

SECURITY REVIEW

SUPERFORM



GETRECON.XYZ
@GALLODASBALLO
@NICANOR

Superform Security Review

Introduction

Alex The Entrepreneur performed a 3 weeks Security Review of Superform in parallel with a 4 weeks Invariant Testing Engagement lead by Nican0r

Repos: <https://github.com/superform-xyz/v2-periphery/tree/dev/src>

This review uses [Code4rena Severity Classification](#)

The Manual Review is done as a best effort service, while a lot of time and attention was dedicated to the security review, it cannot guarantee that no bug is left

As a general rule we always recommend doing one additional security review until no bugs are found, this in conjunction with a Guarded Launch and a Bug Bounty can help further reduce the likelihood that any specific bug was missed

Given the number of integrational issues as well as the extensive changes that happened post review.

I recommend performing another short (1 week / 1.5 weeks) solo review with another lead auditor (a Differential Review of the changes should help give more confidence in them), followed up by a contest (which should include every contract in scope as well as their configuration). This is due to the tight coupling between various contracts (e.g. Oracles) and YieldSources. Any review that doesn't analyze the direct tokens, swappers, hooks, oracles and yield sources is likely to miss compositional risk.

About Recon

Recon offers boutique security reviews, invariant testing development and is pioneering Cloud Fuzzing as a best practice by offering Recon Pro, the most complete tool to run tools such as Echidna, Medusa, Foundry, Kontrol and Halmos in the cloud with just a few clicks

About Alex

Alex is the cofounder of Recon and a well known Lead Security Researcher with \$500,000 in bounties and contest winnings.

- Code4rena - One of the most prolific and respected judges, won the Tapioca contest, at the time the 3rd highest contest pot ever
- Spearbit - Have done reviews for Tapioca, Threshold USD, Velodrome and more
- Recon - Centrifuge Invariant Testing Suite, Corn and Badger invariants as well as live monitoring

Additional Services by Recon

Recon offers:

- Audits powered by Invariant Testing - We'll write your invariant tests then perform an audit on your code.
- Cloud Fuzzing as a Service - The easiest way to run invariant tests in the cloud - Ask about Recon Pro.
- Invariant Tests writing - An engineer will write Chimera based Invariant Tests on your codebase.

High Severity

Issue	Mitigated	PR Resolution
H-01 ERC7540YieldSourceOracle is discounting assets for which requestRedeem is being called	✓	https://github.com/superform-xyz/v2-periphery/pull/118
H-02 ERC7540YieldSourceOracle is not pricing the SuperVaultStrategy shares, causing it to be unable to withdraw the correct amount of assets	✓	https://github.com/superform-xyz/v2-periphery/pull/118
H-03 manager can escape slashing since withdrawStake has no cooldown	✓	https://github.com/superform-xyz/v2-periphery/pull/71

Medium Severity

Issue	Mitigated	PR Resolution
M-01 ERC4626YieldOracle getShareOutput will return the wrong value for a redemption when the vault charges a fee on deposit and not on redemption	✓	https://github.com/superform-xyz/v2-periphery/pull/141
M-02 Sandwiching a step-wise Oracle Update will cause losses to next depositor if they deposit while the PPS is incorrectly overstated	ACK	
M-03 _getPreviousHookOutAmount comparison vs current expectedAssets0rSharesOut can cause unintended reverts	✓	https://github.com/superform-xyz/v2-periphery/pull/84
M-04 Hooks that return a 0 on getOutAmount are incompatible with _processSingleHookExecution	✓	https://github.com/superform-xyz/v2-periphery/pull/81
M-05 ECDSAPPSOracle _forwardValidEntries try/catch can be made to fail due to going OOG, burning the nonce without updating the price	✓	https://github.com/superform-xyz/v2-periphery/pull/72/files
M-06 _getQuoteFromOracle Fetching Chainlink Price Feed with try / catch can be purposefully made to fail by passing insufficient gas	✓	https://github.com/superform-xyz/v2-periphery/pull/92
M-07 _calculateAndTransferFee rounds the total fees down, making it economically feasible to structure withdrawals for low decimal tokens to sidestep the fee	✓	https://github.com/superform-xyz/v2-periphery/pull/55
M-08 Some shares cannot be made withdrawable if processedShares is ever greater than totalRequestedAmount	✓	https://github.com/superform-xyz/v2-periphery/commit/7054abfffa9bc06baeca17bdcfb8959c7e7ff514
M-09 Permissionless Oracle Update in SuperOracleBase likely to cause MEV	✓	https://github.com/superform-xyz/v2-periphery/pull/85
M-10 Automatic Unpausing is dangerous when 2 aberrant prices are broadcasted in a row	✓	https://github.com/superform-xyz/v2-periphery/pull/72/files
M-11 Losses in execution of withdrawal are not imputed to the user that will receive the assets	NUANCED	✓
M-12 Instant mint / deposit in SuperVault allows to front-run yield updates	ACK	
M-13 emergencyPrice being used in a try catch without enforcing min gas could be triggered by an attacker	✓	https://github.com/superform-xyz/v2-periphery/pull/92
M-14 Malicious Main Manager can prevent from being removed by spam creating secondaryManager	✓	https://github.com/superform-xyz/v2-periphery/pull/37

QA (Low and Informational)

Issue	Mitigated	PR Resolution
Q-01 <code>Vault.withdraw</code> <code>shares</code> should <code>roundUp</code>	✓	https://github.com/superform-xyz/v2-periphery/pull/101
Q-02 <code>previewDeposit</code> and <code>previewMint</code> divergence	✓	https://github.com/superform-xyz/v2-periphery/pull/143
Q-03 <code>manageEmergencyWithdraw</code> phase 2 is not necessary and can be removed	✓	https://github.com/superform-xyz/v2-periphery/pull/77
Q-04 Non protected keepers can be made to not be paid out upkeep fees by adding them as <code>authorizedCallers</code>	ACK	
Q-05 <code>_isExemptFromUpkeep</code> is unused	✓	https://github.com/superform-xyz/v2-periphery/pull/86
Q-06 <code>SuperVaultAggregator</code> allows for misleading vault names	✓	https://github.com/superform-xyz/v2-periphery/pull/78
Q-07 <code>previewMint</code> return value with 100% fees	ACK	
Q-08 Small Precision Loss in <code>SuperOracleBase</code> due to separating the computation to have two divisions	✓	https://github.com/superform-xyz/v2-periphery/pull/72
Q-09 Fulfillment reverts if funds haven't been deposited into yield strategy	✓	https://github.com/superform-xyz/v2-periphery/pull/141
Q-10 Collection of QA items for <code>SuperBank</code>	✓	https://github.com/superform-xyz/v2-periphery/pull/61
Q-11 Collection of QA items for <code>ECDSAPP5Oracle</code>	✓	https://github.com/superform-xyz/v2-periphery/pull/100
Q-12 Collection of QA items for <code>SuperGovernor</code>	✓	https://github.com/superform-xyz/v2-periphery/pull/60
Q-13 You can <code>withdraw</code> some assets and "burn" 0 shares due to incorrect rounding direction	✓	https://github.com/superform-xyz/v2-periphery/pull/101/files
Q-14 <code>_processRedeemFulfillments</code> can be grieved via front-running	ACK	
Q-15 <code>_getSlippageTolerance</code> is generally not necessary	✓	https://github.com/superform-xyz/v2-periphery/pull/91
Q-16 <code>distribute</code> should <code>roundUp</code> <code>supAmount</code>	✓	https://github.com/Recon-Fuzz/superform-review/issues/27
Q-17 The <code>SuperBank</code> may not be able to receive funds from contracts that transfer funds with some <code>callData</code>	ACK	
Q-18 Incorrect extra space in <code>UPDATE_PPS_TYPEHASH</code>	✓	https://github.com/superform-xyz/v2-periphery/pull/96
Q-19 <code>Hook.call</code> is still copying the <code>returnData</code> into memory	✓	https://github.com/superform-xyz/v2-periphery/pull/88
Q-20 Inconsistent Merkle Roots generation for <code>Bank</code> hooks	✓	https://github.com/superform-xyz/v2-periphery/pull/80
Q-21 <code>slashStake</code> not callable	✓	https://github.com/superform-xyz/v2-periphery/pull/65

Table of Contents

• High

- H-01 `ERC7540YieldSourceOracle` is discounting assets for which `requestRedeem` is being called
- H-02 `ERC7540YieldSourceOracle` is not pricing the `SuperVaultStrategy` shares, causing it to be unable to withdraw the correct amount of assets
- H-03 `manager` can escape slashing since `withdrawStake` has no cooldown

• Med

- M-01 `ERC4626YieldOracle` `getShareOutput` will return the wrong value for a redemption when the vault charges a fee on deposit and not on redemption
- M-02 Sandwhiching a step-wise Oracle Update will cause losses to next depositor if they deposit while the PPS is incorrectly overstated
- M-03 `_getPreviousHookOutAmount` comparison vs current `expectedAssetsOrSharesOut` can cause unintended reverts
- M-04 Hooks that return a 0 on `getOutAmount` are incompatible with `_processSingleHookExecution`
- M-05 `ECDSAPP5Oracle` `_forwardValidEntries` `try/catch` can be made to fail due to going OOG, burning the nonce without updating the price
- M-06 `_getQuoteFromOracle` Fetching Chainlink Price Feed with `try / catch` can be purposefully made to fail by passing insufficient gas

- M-07 `_calculateAndTransferFee` rounds the total fees down, making it economically feasible to structure withdrawals for low decimal tokens to sidestep the fee
- M-08 Some shares cannot be made withdrawable if `processedShares` is ever greater than `totalRequestedAmount`
- M-09 Permissionless Oracle Update in `SuperOracleBase` likely to cause MEV
- M-10 Automatic Unpausing is dangerous when 2 aberrant prices are broadcasted in a row
- M-11 Losses in execution of withdrawal are not imputed to the user that will receive the assets
- M-12 Instant `mint` / `deposit` in `SuperVault` allows to front-run yield updates
- M-13 `emergencyPrice` being used in a try catch without enforcing min gas could be triggered by an attacker
- M-14 Malicious Main Manager can prevent from being removed by spam creating `secondaryManager`

• QA

- Q-01 `Vault.withdraw` `shares` should roundUp
- Q-02 `previewDeposit` and `previewMint` divergence
- Q-03 `manageEmergencyWithdraw` phase 2 is not necessary and can be removed
- Q-04 Non protected keepers can be made to not be paid out upkeep fees by adding them as `authorizedCallers`
- Q-05 `_isExemptFromUpkeep` is unused
- Q-06 `SuperVaultAggregator` allows for misleading vault names
- Q-07 `previewMint` return value with 100% fees
- Q-08 Small Precision Loss in `SuperOracleBase` due to separating the computation to have two divisions
- Q-09 Fulfillment reverts if funds haven't been deposited into yield strategy
- Q-10 Collection of QA items for `SuperBank`
- Q-11 Collection of QA items for `ECDSAPPSOracle`
- Q-12 Collection of QA items for `SuperGovernor`
- Q-13 You can `withdraw` some assets and "burn" 0 shares due to incorrect rounding direction
- Q-14 `_processRedeemFulfillments` can be grieved via front-running
- Q-15 `_getSlippageTolerance` is generally not necessary
- Q-16 `distribute` should roundUp `supAmount`
- Q-17 The `SuperBank` may not be able to receive funds from contracts that transfer funds with some calldata
- Q-18 Incorrect extra space in `UPDATE_PPS_TYPEHASH`
- Q-19 `Hook.call` is still copying the `returnData` into memory
- Q-20 Inconsistent Merkle Roots generation for `Bank` hooks
- Q-21 `slashStake` not callable

• Invariants

- I-01 Return value of `maxRedeem` is incorrect
- I-02 Invariant Testing Suggested Next Steps
- I-03 Users can lose value on mint/redeem
- I-04 Invariant Testing Result
- I-05 Unbacked shares remain on withdrawal when withdrawals cause insolvency
- I-06 Insolvency in `SuperVaultStrategy`
- I-07 Attempting to withdraw `maxWithdraw` reverts
- I-08 `fulfillRedeemRequests` can cause `accumulatorShares` to not decrease correctly
- I-09 Donations to yield source cause a loss on withdrawal for users
- I-10 Remainder optimization tests
- I-11 Interesting Preconditions

• Analysis

- A-01 Tokens for which a insolvency tolerance may be necessary
- A-02 Incomplete Notes on accounting/oracles

- A-03 Suggested Next Steps
- A-04 Nuanced Risk that comes from Pricing via the Pricing Oracle - With emphasis on how to Safely Price ERC4626
- A-05 Superficial discussion on Risks and Complexities of PPS being determine by a group of validators
- A-06 Collection of Admin Related Risks
- A-07 Collection of Operative Risks
- A-08 Some gotcha tied to PPS being aberrant and vault paused - Likely totalAssets would then also be a dangerous value to read for integrators
- A-09 Incomplete List of Gotchas related to the Integration Targets of the SuperVault
- **Economic**
 - E-01 Deposits and Withdraw will benefit by having a Bid and Ask Pricing

H-01 `ERC7540YieldSourceOracle` is discounting assets for which `requestRedeem` is being called

Impact

`ERC7540YieldSourceOracle.getTVLByOwnerOfShares` looks as follows:

<https://github.com/superform-xyz/v2-core/blob/fcce91733c1fa9c63dc8bb3b1c9a75930fd65003/src/accounting/oracles/ERC7540YieldSourceOracle.sol#L75-L88>

```
/// @inheritdoc AbstractYieldSourceOracle
function getTVLByOwnerOfShares(
    address yieldSourceAddress,
    address ownerOfShares
)
    public
    view
    override
    returns (uint256)
{
    uint256 shares = IERC20(IERC7540(yieldSourceAddress).share()).balanceOf(ownerOfShares);
    if (shares == 0) return 0;
    return IERC7540(yieldSourceAddress).convertToAssets(shares);
}
```

This is estimating the TVL controller by `ownerOfShares`

But because ERC7540 is asynchronous, the TVL can also include assets that can be claimed via `redeem` / `withdraw`, in the case of Centrifuge V3, the assets are exactly `maxWithdraw`.

Additionally shares that have yet to be redeemed, will be moved to the escrow.

POC

The following POC is highly speculative as I don't have context on how this function will be used

- Have X assets
- Queue a withdrawal on the ERC7540
- Assets depress since the `getTVLByOwnerOfShares` will underestimate the assets (the shares are in the escrow)
- Redemption is fulfilled on centrifuge side
- Assets are now in `maxWithdraw` but the `getTVLByOwnerOfShares` return value is still depressed
- New deposits will be able to mint shares at this depressed price (assuming `getTVLByOwnerOfShares` is involved in how PPS is computed)

Mitigation

The ERC7540 oracle will have to be vault specific And will have to account for requested redemptions As well as redemptions fulfilled

H-02 `ERC7540YieldSourceOracle` is not pricing the `SuperVaultStrategy` shares, causing it to be unable to withdraw the correct amount of assets

Impact

`_processSingleFulfillHookExecution` will use `getShareOutput` from the `IYieldSourceOracle` to determine how many shares to withdraw from the target

<https://github.com/superform-xyz/v2-periphery/blob/abbf22cd06a7c508dc2cf2974208fb0aeec61ed2/src/SuperVault/SuperVaultStrategy.sol#L666-L673>

```
// Calculate underlying shares and update hook calldata
vars.amountOfAssets = vars.superVaultShares.mulDiv(currentPPS, PRECISION, Math.Rounding.Floor);
vars.svAsset = address(_asset); /// @audit Centrifuge Grief / Mistake based on how the oracle is implemented?
vars.amountConvertedToUnderlyingShares = IYieldSourceOracle(yieldSources[vars.targetedYieldSource])
    .getShareOutput(vars.targetedYieldSource, vars.svAsset, vars.amountOfAssets);
hookCalldata =
    ISuperHookOutflow(hook).replaceCalldataAmount(hookCalldata, vars.amountConvertedToUnderlyingShares);
```

In the case of ERC7540, the implementation for the `ERC7540YieldSourceOracle` is the following:

<https://github.com/superform-xyz/v2-core/blob/dev/src/accounting/oracles/ERC7540YieldSourceOracle.sol>

```
/// @inheritdoc AbstractYieldSourceOracle
function getShareOutput(
    address yieldSourceAddress,
    address,
    uint256 assetsIn
)
    external
    view
    override
    returns (uint256)
{
    return IERC7540(yieldSourceAddress).convertToShares(assetsIn);
}
```

In the case of Centrifuge V3, `convertToShares` will use the Global Price that the vault has:

<https://github.com/centrifuge/protocol-v3/blob/4f69794392c44cdbf6d27121fd37a2c6ec9eeced/src/vaults/SyncManager.sol#L160-L178>

```
/// @inheritdoc ISyncManager
function convertToShares(IBaseVault vault_, uint256 assets) public view returns (uint256 shares) {
    VaultDetails memory vaultDetails = spoke.vaultDetails(vault_);

    D18 poolPerShare = pricePoolPerShare(vault_.poolId(), vault_.scId());
    D18 poolPerAsset = spoke.pricePoolPerAsset(vault_.poolId(), vault_.scId(), vaultDetails.assetId, true);

    return poolPerShare.isZero()
        ? 0
        : PricingLib.assetToShareAmount(
            vault_.share(),
            vaultDetails.asset,
            vaultDetails.tokenId,
            assets.toUint128(),
            poolPerAsset,
            poolPerShare,
            MathLib.Rounding.Down
        );
}
```

Whereas when performing a redemption/withdrawal, the Centrifuge `AsyncRequestManager` will use the `state.redeemPrice` which is custom for each controller (in the case the `SuperVaultStrategy`)

<https://github.com/centrifuge/protocol-v3/blob/4f69794392c44cdbf6d27121fd37a2c6ec9eeded/src/vaults/AsyncRequestManager.sol#L368-L397>

```
/// @inheritdoc IRedeemManager
function redeem(IBaseVault vault_, uint256 shares, address receiver, address controller)
    public
    auth
    returns (uint256 assets)
{
    _checkIsLinked(vault_);
    require(shares <= maxRedeem(vault_, controller), ExceedsMaxRedeem());

    AsyncInvestmentState storage state = investments[vault_][controller];

    uint128 assetsUp = _shareToAssetAmount(vault_, shares.toUint128(), state.redeemPrice, MathLib.Rounding.Up);
    uint128 assetsDown = _shareToAssetAmount(vault_, shares.toUint128(), state.redeemPrice,
MathLib.Rounding.Down);
    _processRedeem(state, assetsUp, assetsDown, vault_, receiver, controller);
    assets = uint256(assetsDown);
}

/// @inheritdoc IRedeemManager
function withdraw(IBaseVault vault_, uint256 assets, address receiver, address controller)
    public
    auth
    returns (uint256 shares)
{
    _checkIsLinked(vault_);

    AsyncInvestmentState storage state = investments[vault_][controller];
    uint128 assets_ = assets.toUint128();
    _processRedeem(state, assets_, assets_, vault_, receiver, controller);

    shares = uint256(_assetToShareAmount(vault_, assets_, state.redeemPrice, MathLib.Rounding.Up));
}
```

This will break the accounting having the following impacts:

POC

When the PPS of the Centrifuge Vault is higher

- Withdraws less shares than the price that the SuperVaultStrategy has for it's withdrawals
- Causes direct losses to withdrawers

When the PPS of the Centrifuge Vault is lower

- Withdraws too many shares
- Causes a direct loss to the SuperVaultStrategy
- Can revert due to attempting to withdraw more than the shares available

Note that the total shares available for withdrawal/redemption is a virtual number computed by using `maxWithdraw` and `state.redeemPrice` meaning an additional rounding factor has to be taken into account.

Mitigation

You likely will have to write a specific oracle for each target (as each ERC7540 will have a different way of storing withdrawal prices) And make sure to use the correct price.

In this case you'll need to compute the shares by using:

```
mapping(IBaseVault vault => mapping(address investor => AsyncInvestmentState)) public investments;
```

with the formula:

```
function _assetToShareAmount(IBaseVault vault_, uint128 assets, D18 priceAssetPerShare, MathLib.Rounding rounding)
    internal
    view
    returns (uint128 shares)
{
    VaultDetails memory vaultDetails = spoke.vaultDetails(vault_);
    address shareToken = vault_.share();

    return priceAssetPerShare.isZero()
        ? 0
        : PricingLib.assetToShareAmount(
            shareToken, vaultDetails.asset, vaultDetails.tokenId, assets, priceAssetPerShare, rounding
        );
}
```

Applying rounding down as to cause the slight loss to the recipient instead of the `SuperVaultStrategy`

H-03 `manager` can escape slashing since `withdrawStake` has no cooldown

Impact

At this time it seems that slashing is not available since `slashStake` cannot be called.

However, the design of the code allows to escape a slashing

Since `withdrawStake` has no delay

<https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultAggregator.sol#L402-L423>

```
function withdrawStake(uint256 amount) external {
    if (amount == 0) revert ZERO_ADDRESS(); // Reusing error code for consistency

    // Check sufficient balance
    if (_managerStakeBalance[msg.sender] < amount) {
        revert INSUFFICIENT_STAKE_BALANCE();
    }

    // Get the UP token address from SUPER_GOVERNOR
    address upToken = SUPER_GOVERNOR.getAddress(SUPER_GOVERNOR.UP());

    // Update stake balance
    unchecked {
        _managerStakeBalance[msg.sender] -= amount;
    }

    // Transfer UP tokens to manager
    IERC20(upToken).safeTransfer(msg.sender, amount);

    emit StakeWithdrawn(msg.sender, amount);
}
```

A manager that is about to get slashed can simply call `withdrawStake` to avoid getting slashed

Mitigation

`withdrawStake` should be behind a timelock that allows the governor to perform slashing, if `slashStake` will have a delay then the delay for `withdrawStake` must be long enough to allow slashing

Additionally, stake that is queued for withdrawal should no longer be considered valid as otherwise a manager could simply queue a withdrawal so that they have the ability to evade slashing while retaining their stake

M-01 ERC4626YieldOracle `getShareOutput` will return the wrong value for a redemption when the vault charges a fee on deposit and not on redemption

Impact

The function `getShareOutput` in `ERC4626YieldOracle` looks as follows:

<https://github.com/superform-xyz/v2-core/blob/fcce91733c1fa9c63dc8bb3b1c9a75930fd65003/src/accounting/oracles/ERC4626YieldSourceOracle.sol#L25-L37>

```
/// @inheritdoc AbstractYieldSourceOracle
function getShareOutput(
    address yieldSourceAddress,
    address,
    uint256 assetsIn
)
    external
    view
    override
    returns (uint256)
{
    return IERC4626(yieldSourceAddress).previewDeposit(assetsIn);
}
```

This is using `previewDeposit` which in a ERC4626 compliant vault will include fees and possible slippage.

The function is used to determine how many shares to redeem from the underlying vault

<https://github.com/superform-xyz/v2-periphery/blob/597f98364b10863c87acb43d689ab932856777d8/src/SuperVault/SuperVaultStrategy.sol#L629-L634>

```
vars.amountOfAssets = IERC4626(_vault).convertToAssets(vars.superVaultShares);
vars.svAsset = address(_asset);
vars.amountConvertedToUnderlyingShares = IYieldSourceOracle(yieldSources[vars.targetedYieldSource])
    .getShareOutput(vars.targetedYieldSource, vars.svAsset, vars.amountOfAssets);
hookCalldata =
    ISuperHookOutflow(hook).replaceCalldataAmount(hookCalldata, vars.amountConvertedToUnderlyingShares);
```

So, when `deposit` of the underlying vault has a fee, but redeeming doesn't this will cause a loss to the `controller`, although no loss has happened.

Mitigation

You should review each Vault integration and ensure that non of these gotchas are possible. I believe that for Euler, Morpho and Yearn, this risk is not present

M-02 Sandwiching a step-wise Oracle Update will cause losses to next depositor if they deposit while the PPS is incorrectly overstated

Impact

As discussed by the Superform Team: <https://www.notion.so/superform/MEV-Protection-Research-and-Discuss-Approach-27e35672200c806a9535ecdd5efefdc2> and discussed in this review, a deposit can sandwich an oracle update, to steal the yield.

Doing that has an additional side effect, the PPS will be overstated.

Depositing against that overstated PPS will cause losses to depositors.

The losses will be permanent until the PPS is updated to accurately reflect the ratio between the Assets and the Shares.

POC

-> Assume the Vault has a 1 to 1 (1e18 PPS -> Deposits will receive 1 to 1 shares for assets -> Say we will have a drastic improvement in the PPS, to 1.5 -> MEV Actor frontruns the update, the "real" PPS should be 1.25. But onchain we have 2x the shares at 1.5 PPS (overstated PPS) -> Victim deposits at 1.5, they are overpaying by $(1.5 / 1.25 = 20\%)$, they do not lock in any losses now as the PPS is still 1.5 -> The PPS is updated to 1.25, which should have been the correct ratio (arguably the PPS may be different if we account for the assets / shares of victim, but that is beside the point). Victim has locked in a 20% loss through no fault of their own

Mitigation

Any time the update to the PPS will be small, this should not be an issue.

However, at times where the PPS will update significantly, it may be necessary to ensure that the vault was not sandwiched, perhaps by performing a onchain check of the total assets of the vault + strategy.

An alternative fix is to vest these changes over time.

Another plausible alternative is that if a PPS update will cause spikes, you could temporarily raise the minting fees until the Price has updated.

M-03 `\_getPreviousHookOutAmount` comparison vs current `expectedAssetsOrSharesOut` can cause unintended reverts

Impact

`_processSingleHookExecution` looks as follows:

<https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L594-L617>

```
function _processSingleHookExecution(
    address hook,
    address prevHook,
    bytes memory hookCalldata,
    uint256 expectedAssetsOrSharesOut
)
    internal
    returns (address)
{
    ExecutionVars memory vars;
    vars.hookContract = ISuperHook(hook);

    vars.targetedYieldSource = HookDataDecoder.extractYieldSource(hookCalldata);
    if (yieldSources[vars.targetedYieldSource] == address(0)) revert YIELD_SOURCE_NOT_FOUND();

    bool usePrevHookAmount = _decodeHookUsePrevHookAmount(hook, hookCalldata);

    if (usePrevHookAmount && prevHook != address(0)) {
        vars.outAmount = _getPreviousHookOutAmount(prevHook);
        if (expectedAssetsOrSharesOut == 0) revert ZERO_EXPECTED_VALUE();

        uint256 minExpectedPrevOut = expectedAssetsOrSharesOut * (BPS_PRECISION - _getSlippageTolerance());
        if (vars.outAmount * BPS_PRECISION < minExpectedPrevOut) {
            revert MINIMUM_PREVIOUS_HOOK_OUT_AMOUNT_NOT_MET();
        }
    }
}
```

The check is using the current hooks `expectedAssetsOrSharesOut` and comparing the previous hook return value, taken from `_getPreviousHookOutAmount`

This can creates reverts in scenarios in which some conversion is happening.

POC

Hook 1

If I convert WETH to wstETH I will go from say 1e18 to .8e18

I can specify this as `expectedAssetsOrSharesOut` to be .8e18

The check is performed after the hook call, and passes.

Hook 2

Say I deposit the wstETH to a vault and I'm now expecting to receive 1e18 shares of the wstETH denominated vault

I would reasonably set the `expectedAssetsOrSharesOut` to be 1e18

Since I need to use the amount to deposit from the previous hook, I'll likely have enabled `usePrevHookAmount` and will retrieve the value .8e18 from `_getPreviousHookOutAmount`

My slippage for Hook 2 is set to 1e18

This will cause the revert: `MINIMUM_PREVIOUS_HOOK_OUT_AMOUNT_NOT_MET`

Even though the original slippage check was fine

Mitigation

I believe each hooks return value should be validated independently

If you wish to validate the `_getPreviousHookOutAmount` then you should introduce a new parameter called `expectedPrevHookAmount` which will ensure you can handle these cases

M-04 Hooks that return a 0 on `getOutAmount` are incompatible with `\_processSingleHookExecution`

Impact

Hook executions is done via `_processSingleHookExecution`

The function has the following check:

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L626-L630)

[periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L626-L630](https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L626-L630)

```
ISuperHook(address(vars.hookContract)).resetExecutionState(address(this));

uint256 actualOutput = ISuperHookResult(hook).getOutAmount(address(this));
if (actualOutput == 0) revert ZERO_OUTPUT_AMOUNT();
```

The check will revert for any hook that doesn't update the `getOutAmount` to a non zero value.

Instances Found (may be incomplete)

Merkle `_setOutAmount` is set to 0 in the `MerkleClaimRewardHook`

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-core/blob/780ad7daa523feee612e1fd935028c1f2671b261/src/hooks/claim/merkle/MerkleClaimRewardHook.sol#L129-L131)

[core/blob/780ad7daa523feee612e1fd935028c1f2671b261/src/hooks/claim/merkle/MerkleClaimRewardHook.sol#L129-L131](https://github.com/superform-xyz/v2-core/blob/780ad7daa523feee612e1fd935028c1f2671b261/src/hooks/claim/merkle/MerkleClaimRewardHook.sol#L129-L131)

```
function _postExecute(address, address account, bytes calldata) internal override {
    _setOutAmount(0, account);
}
```

Some Morpho hooks

M-05 `ECDSAPPSOracle` `\_forwardValidEntries` `try/catch` can be made to fail due to going OOG, burning the nonce without updating the price

Impact

`_forwardValidEntries` in the newer version of `ECDSAPPSOracle` looks as follows:

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/ECDSAPPSOracle.sol#L264-L291)

[periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/ECDSAPPSOracle.sol#L264-L291](https://github.com/superform-xyz/v2-periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/ECDSAPPSOracle.sol#L264-L291)

```
function _forwardValidEntries(
    address[] memory validStrategies,
    uint256[] memory validPpss,
    uint256[] memory validPpsStdevs,
    uint256[] memory validValidatorSets,
    uint256[] memory validTotalValidators,
    uint256[] memory validTimestamps
) internal {
    // Only forward if there are valid entries
    if (validStrategies.length > 0) {
        try ISuperVaultAggregator(SUPER_GOVERNOR.getAddress(SUPER_VAULT_AGGREGATOR)).forwardPPS(
            ISuperVaultAggregator.ForwardPPSArgs({
                strategies: validStrategies,
                ppss: validPpss,
                ppsStdevs: validPpsStdevs,
                validatorSets: validValidatorSets,
                totalValidators: validTotalValidators,
                timestamps: validTimestamps,
                updateAuthority: msg.sender
            })
        ) {
        } catch Error(string memory reason) {
            emit BatchForwardPPSFailed(reason);
        } catch (bytes memory lowLevelData) {
            emit BatchForwardPPSFailedLowLevel(lowLevelData);
        }
    }
}
```

The try catch is in place as the aggregator can revert due to validation / staleness check.

However, this check could revert due to going OOG on a valid PPS Update.

This would burn the nonce without updating the price.

Mitigation

I believe you have to include an extra check before the `try` that ensures that the call will be performed with sufficient gas

In lack of that the call could revert due to going OOG

M-06 `\_getQuoteFromOracle` Fetching Chainlink Price Feed with `try / catch` can be purposefully made to fail by passing insufficient gas

Impact

`_getQuoteFromOracle` performs a call to the Chainlink Aggregator Proxy in the follow way:

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/SuperOracleBase.sol#L323-L331)

[periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/SuperOracleBase.sol#L323-L331](https://github.com/superform-xyz/v2-periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/SuperOracleBase.sol#L323-L331)

```
try AggregatorV3Interface(oracle).latestRoundData() returns (
    uint80, int256 _answer, uint256, uint256 _updatedAt, uint80
) {
    answer = _answer;
    updatedAt = _updatedAt;
} catch {
    if (revertOnError) revert ORACLE_ROUND_DATA_CALL_FAIL(oracle);
    return 0;
}
```

This can be made to revert on purpose by passing a low amount of gas

Mitigation

See how Liquity mitigated this.

The check has been triple audited + passed more than 1 security contest (Liquity and Asymmetry Finance)

<https://github.com/liquity/bold/blob/d333d1332b8ee4c82f8528edf058eef927eb6fdf/contracts/src/PriceFeeds/MainnetL105>

M-07 `\_calculateAndTransferFee` rounds the total fees down, making it economically feasible to structure withdrawals for low decimal tokens to sidestep the fee

Impact

The code for `_calculateAndTransferFee` looks as follows:

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L805-L816>

```
function _calculateAndTransferFee(
    uint256 currentAssetsWithFees,
    uint256 historicalAssets
)
private
returns (uint256 currentAssets)
{
    currentAssets = currentAssetsWithFees;
    if (currentAssetsWithFees > historicalAssets) {
        uint256 profit = currentAssetsWithFees - historicalAssets;
        uint256 performanceFeeBps = feeConfig.performanceFeeBps;
        uint256 totalFee = profit.mulDiv(performanceFeeBps, BPS_PRECISION, Math.Rounding.Floor); // @audit Rounded
    }
}
```

The `totalFee` is rounded down

This means that there exist an amount of profit for which the fee is zero.

As long as $\text{profit} * \text{feeBPS} / 10_000$ is zero, then the fee will be skipped.

For most tokens with 18 decimals this is not an issue.

However, for tokens with 6 decimals $1/10_000$ is 1 cent and for tokens with 2 decimals (gUSD) this can be 100 dollars.

For these tokens, especially in L2, it can be economically feasible to consume the gas necessary to structure withdrawals into small amounts, as to sidestep the fee.

Mitigation

Change the fee on redemption to roundUp

This will make it so that the fee is at least 1 wei, making these type of considerations no longer relevant

M-08 Some shares cannot be made withdrawable if ``processedShares`` is ever greater than ``totalRequestedAmount``

Impact

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L672-L688>

```
function _processRedeemFulfillments(
    address[] calldata controllers,
    uint256 controllersLength,
    uint256 processedShares,
    uint256 currentPPS
)
internal
{
    uint256 totalRequestedAmount = 0;
    uint256[] memory controllerRequestedAmount = new uint256[](controllersLength);
    for (uint256 i; i < controllersLength; ++i) {
        controllerRequestedAmount[i] = superVaultState[controllers[i]].pendingRedeemRequest;
        totalRequestedAmount += controllerRequestedAmount[i];
    }
    if (processedShares + TOLERANCE_CONSTANT < totalRequestedAmount) {
        revert INVALID_REDEEM_FILL();
    }
}
```

Where:

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L634-L639>

```
// we must always encode supervault shares when fulfilling redemptions
vars.superVaultShares = ISuperHookInflowOutflow(hook).decodeAmount(hookCalldata);

// Calculate underlying shares and update hook calldata
vars.amountOfAssets = vars.superVaultShares.mulDiv(currentPPS, PRECISION, Math.Rounding.Floor);
vars.svAsset = address(_asset);
```

And then returned: <https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L664-L665>

```
return vars.superVaultShares;
```

And accumulated: <https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L302>

```
processedShares += amountSharesSpent;
```

Meaning we must specify the shares that we want to process for each user in order for them to be withdrawable.

The simplest example being having one user that needs to receive 100% of their shares.

This is always feasible with the exception of the case in which we redeem more shares than the sum of requests.

Whenever this happens, we will then have insufficient shares in the escrow (which needs to be burned exactly), and we won't be able to set the shares to be redeemed for the user to their exact requested amount.

This may happen exclusively as an admin mistake.

Mitigation

However, it seems to me that this check should be an exact check:

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L686-L688>

```
if (processedShares + TOLERANCE_CONSTANT < totalRequestedAmount) {  
    revert INVALID_REDEEM_FILL();  
}
```

M-09 Permissionless Oracle Update in `SuperOracleBase` likely to cause MEV

Impact

The Governor can queue an oracle update, with the handling code looking as follows:

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/oracles/SuperOracleBase.sol#L120-L151>

```
function queueOracleUpdate(
    address[] calldata bases,
    address[] calldata quotes,
    bytes32[] calldata providers,
    address[] calldata feeds
)
    external
{
    if (msg.sender != SUPER_GOVERNOR) revert UNAUTHORIZED_UPDATE_AUTHORITY();
    uint256 length = bases.length;
    if (length != quotes.length || length != providers.length || length != feeds.length) {
        revert ARRAY_LENGTH_MISMATCH();
    }

    _validateOracleInputs(bases, quotes, providers, feeds);

    pendingUpdate = PendingUpdate({
        bases: bases,
        quotes: quotes,
        providers: providers,
        feeds: feeds,
        timestamp: block.timestamp
    });

    emit OracleUpdateQueued(bases, quotes, providers, feeds, block.timestamp);
}

/// @inheritdoc ISuperOracle
function executeOracleUpdate() external { /// @audit Likely to leak MEV since it's permissionless.
    if (pendingUpdate.timestamp == 0) revert NO_PENDING_UPDATE();
    if (block.timestamp < pendingUpdate.timestamp + TIMELOCK_PERIOD) revert TIMELOCK_NOT_ELAPSED();
}
```

When an update is ready, anyone can call `executeOracleUpdate`

Assuming feeds and providers are any "normal" oracle.

The permissionless nature is likely to create a step-wise change (jump) in how assets are priced.

This step-wise change likely will be an opportunity to either receive more shares than intended or have them appreciate instantly

Mitigation

It's best to protect this function to avoid leaking value to MEV actors at the detriment of other depositors

M-10 Automatic Unpausing is dangerous when 2 aberrant prices are broadcasted in a row

Impact

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultAggregator.sol#L1051-L1060>

```
// Pause strategy if any check failed
if (checksFailed && !_strategyData[args.strategy].isPaused) {
    _strategyData[args.strategy].isPaused = true;
    emit StrategyPaused(args.strategy);
}
// Unpause strategy if all checks passed and strategy was previously paused
else if (!checksFailed && _strategyData[args.strategy].isPaused) { /// @audit likely not ideal
    _strategyData[args.strategy].isPaused = false;
    emit StrategyUnpaused(args.strategy); /// @audit If you update then the next aberrant price will be
considered normal
} /// And the next normal price will be considered aberrant
```

POC

- P0 is a valid Price
- P1 is aberrant price -> It triggers a pause
- P2 is aberrant to P0 but consistent to P1 -> It triggers an unpause
- Deposits and Redemption can now be triggered with the aberrant price
- P3 is a price that is consistent with P0 -> > It triggers a pause
- P4 is the first price at which the Vault can trade again at a valid non aberrant price

Any actor that was able to interact with the vault at P2, will likely have gained some value (deposit front-run), or due to an Manager mistake they may process redemptions with an aberrant price.

Mitigation

I believe you should not unpause on prices that fail the checks But rather have a role to unpause (likely behind a short timelock)

M-11 Losses in execution of withdrawal are not imputed to the user that will receive the assets

Impact

When processing redemptions, the total assets a controller will receive is computed in this way:

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L780-L799>

```
function _processFees(
    uint256 requestedShares,
    uint256 currentPricePerShare,
    uint256 historicalAssets
)
private
returns (uint256 currentAssetsWithFees, uint256 currentAssets)
{
    // Calculate current value of the shares at current price
    currentAssetsWithFees = requestedShares.mulDiv(currentPricePerShare, PRECISION, Math.Rounding.Floor);

    // Apply fees only on profit
    currentAssets = _calculateAndTransferFee(currentAssetsWithFees, historicalAssets);

    // Ensure we don't exceed available balance
    uint256 balanceOfStrategy = _getTokenBalance(address(_asset), address(this));
    currentAssets = currentAssets > balanceOfStrategy ? balanceOfStrategy : currentAssets;

    return (currentAssetsWithFees, currentAssets);
}
```

This is using the `currentPricePerShare` which is not accounting for losses that may have accrued when processing the withdrawal.

The execution of the hook is done in this way:

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L645-L660>

```
vars.balanceAssetBefore = _getTokenBalance(vars.svAsset, address(this));

ISuperHook(address(vars.hookContract)).setExecutionContext(address(this));

vars.executions = vars.hookContract.build(address(0), address(this), hookCalldata);
for (uint256 j; j < vars.executions.length; ++j) {
    (vars.success,) = vars.executions[j].target.call(vars.executions[j].callData);
    if (!vars.success) revert OPERATION_FAILED();
}
ISuperHook(address(vars.hookContract)).resetExecutionState(address(this));

vars.outAmount = _getTokenBalance(vars.svAsset, address(this)) - vars.balanceAssetBefore;

if (vars.outAmount == 0) revert ZERO_OUTPUT_AMOUNT();
if (vars.outAmount * BPS_PRECISION < expectedAssetOutput * (BPS_PRECISION - _getSlippageTolerance())) {
    revert MINIMUM_OUTPUT_AMOUNT_ASSETS_NOT_MET();
}
```

A delta of balances is used, to ensure that slippage is sufficient

However, the delta balance can carry a loss.

Additionally, when withdrawing from vaults (such as ERC4626 or ERC7540), more shares may be burned by the caller to receive a certain amount of assets, causing real losses to the caller (in this cause the SuperVaultStrategy, meaning a loss to all other depositors).

Neither of these 2 losses are imputed to the controllers, locking in losses to idle depositors.

3 sources of losses to the vault

- Normal Loss from Vaults | Should be socialized

Whenever a user calls `Vault.withdraw` most (safe) ERC4626 will roundUp the amount of shares to burn. This typically causes 1 (wei) share to be lost by the caller. The value of the share is typically small, but in some scenario the value can be significant. The mitigation for this is to check the net value of holdings before and after the hook, to ensure that no additional loss has happened.

- Additional Accrued Loss due to Price Per Share not being updated in time | Admin Mistake, Processing should be done around PPS update

Since the mechanism for updating Price Per Share for the vault is asynchronous, withdrawals that are processed with a stale or incorrect (higher than intended) Price Per Share, will cause the controller to receive an old, over-estimated amount, at the detriment of all other depositors. The only mitigation for this is for the Manager to ensure the price is updated, or to update the price before they process redemptions.

- Actual effective loss due to slippage / execution loss | Must not be socialized.

Some Vaults such as the Yearn Vault can accrue losses when withdrawing and redeeming. By default the Yearn vault accepts no losses on withdraw but up to 100% of losses on redemption:

<https://github.com/yearn/yearn-vaults-v3/blob/104a2b233bc6d43ba0720d68355b04d2dc31795/contracts/VaultV3.vy#L1825-L1856>

A manager could also call the function with custom parameters (say 10 BPS loss)

Whenever a loss happens the Yearn Vault would return less assets than requested:

<https://github.com/yarn/yarn-vaults-v3/blob/104a2b233bc6d43ba40720d68355b04d2dc31795/contracts/VaultV3.vy#L867-L869>

While still burning the exact amount of shares requested. Causing a direct loss to the SuperVault.

These losses should be imputed to the controllers, by updating the "fulfilled assets" to be the actual amount.

- Losses due to receiving less assets than the exact amount

Some operations such as swaps can also cause losses, similarly to Yearn V3, in this case the loss should be imputed to the Controllers and not to the Super Vault

Coded POC

Logs:

Assets in Strategy B4 0

Shares in vault B4 0

Max Redeem B4 0

```
_getHookAddressAndCalldata amountToInvest 1980000000000000000000
```

```
decodeAmount 1980000000000000000000
```

Assets in Strategy 0

Shares in vault 1980000000000000000000

Max Redeem 198000000000000000000

pendingRedeemRequest 0

```
pendingRedeemRequest 1
```

Max Withdraw Acc 1881000000000000000000

Strategy Balance Solvency 1782000000000000000000

Max Withdraw 891000000000000000000

Max Withdraw 9900000000000000000000

```

/// @dev Test: Multi-actor deposit, withdrawal request, loss simulation, and distribution validation
function test_multiActorDepositWithdrawLossDistribution() public {
    console2.log("Assets in Strategy B4", MockERC20(superVault.asset()).balanceOf(address(superVaultStrategy)));
    console2.log("Shares in vault B4", MockERC20(_getYieldSource()).balanceOf(address(superVaultStrategy)));
    console2.log("Max Redeem B4", MockERC4626Tester(_getYieldSource()).maxRedeem(address(superVaultStrategy)));

    // Deposit
    superVault_deposit(1000e18);
    switchActor(1);
    superVault_deposit(1000e18);

    switchActor(0); // Back to 0

    // Add yield source
    superVaultStrategy_manageYieldSource_clamped(0);

    // Deposit into it
    uint256[] memory hookTypeInts = new uint256[](1);
    hookTypeInts[0] = 0; // ApproveAndDeposit4626

    uint256[] memory amountsToInvest = new uint256[](1);
    amountsToInvest[0] = MockERC20(superVault.asset()).balanceOf(address(superVaultStrategy));

    bool[] memory usePrevAmounts = new bool[](1);
    usePrevAmounts[0] = false;

    superVaultStrategy_executeHooks_clamped(hookTypeInts, amountsToInvest, usePrevAmounts);

    console2.log("Assets in Strategy", MockERC20(superVault.asset()).balanceOf(address(superVaultStrategy)));
    console2.log("Shares in vault", MockERC20(_getYieldSource()).balanceOf(address(superVaultStrategy)));
    console2.log("Max Redeem", MockERC4626Tester(_getYieldSource()).maxRedeem(address(superVaultStrategy)));

    // // Set loss on withdraw for ERC4626
    MockERC4626Tester(_getYieldSource()).setLossOnWithdraw(1000);

    // Request all redemptions
    superVault_requestRedeem_clamped(superVault.balanceOf(_getActor()));
    switchActor(1);
    superVault_requestRedeem_clamped(superVault.balanceOf(_getActor()));

    switchActor(0);
    superVaultStrategy_fulfillRedeemRequests_clamped(superVaultStrategy.pendingRedeemRequest(_getActor()));
    console2.log("pendingRedeemRequest", "0");
    switchActor(1);
    superVaultStrategy_fulfillRedeemRequests_clamped(superVaultStrategy.pendingRedeemRequest(_getActor()));
    console2.log("pendingRedeemRequest", "1");
    switchActor(0);

    // Compute the insolvency
    uint256 maxWithdrawAcc;
    for (uint256 i; i < _getActors().length; i++) {
        maxWithdrawAcc += superVault.maxWithdraw(_getActors()[i]);
    }

    console2.log("Max Withdraw Acc", maxWithdrawAcc);
    console2.log("Strategy Balance Solvency",
MockERC20(superVault.asset()).balanceOf(address(superVaultStrategy)));

    // Show the revert
    console2.log("Max Withdraw", superVault.maxWithdraw(_getActor()));
    superVault_withdraw(superVault.maxWithdraw(_getActor()));
    switchActor(1);
    console2.log("Max Withdraw", superVault.maxWithdraw(_getActor()));
    superVault_withdraw(superVault.maxWithdraw(_getActor()));

    // The above reverts due to insufficient assets. The Vault gave away too many assets
}

```

Mitigation

The `currentAssetsWithFees` should be computed by determining the assets - the losses accrued when unlocking those assets.

At the end of the operation, the PPS for the Vault should not have decreased (which would indicate a loss to the vault).

Pseudocoded fix

The loop of hook calls should be wrapped around a function that tracks the shares redeemed, the total assets received off of processing said redemptions and the loss that was locked due to that.

```
(uint256 processedShares, uint256 netAssetsWithdrawn, uint256 loss) = _callHooks(...);
```

`_processRedeemFulfillments` should receive the `processedShares` and the `netAssetsWithdrawn` instead of the `currentPPS`

Off of these 2 variables you would then distribute the assets to each recipient without needing to use the price per share (which carries an additional level of precision loss)

M-12 Instant `mint` / `deposit` in `SuperVault` allows to front-run yield updates

Impact

`deposit` looks as follows:

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVault.sol#L138-L155>

```
function deposit(uint256 assets, address receiver) public override nonReentrant returns (uint256 shares) {
    if (receiver == address(0)) revert ZERO_ADDRESS();
    if (assets == 0) revert ZERO_AMOUNT();

    // Forward assets from msg-sender to strategy
    _asset.safeTransferFrom(msg.sender, address(strategy), assets); /// @audit CEI!

    // Single executor call: strategy skims entry fee, accounts on NET, returns net shares
    shares = strategy.handleOperations4626Deposit(receiver, assets);
    if (shares == 0) revert ZERO_AMOUNT();

    // Mint the net shares
    _mint(receiver, shares);

    emit Deposit(msg.sender, receiver, assets, shares);
}
```

It allows to synchronously receive shares for the vault using the cached PPS

This opens up to frontrunning attacks where an actor would wait to detect a positive PPS update in the oracle, frontrun it by calling `deposit` on the vault and then gaining an increase in the PPS right after.

It's worth noting that for integrations that are step-wise in nature (e.g. ERC7540 that pay a premium every month, strategies that have to claim assets and compound them every X days without linear vesting), this attack can become highly profitable.

Due to this it's critical that PPS updates are as smooth as possible.

POC

Invariant testing demonstrates the issue, the naive PPS decreases when a deposit happens, because the actual underlying assets are being socialized to new depositors due to using a hardcoded PPS

```
// forge test --match-test test_property_naivePPSDoesntChangeOnDepositOrMint_ -vvv
function test_property_naivePPSDoesntChangeOnDepositOrMint_() public {

    superVault_mint(1);

    yieldSource_mint(1,0x0000000000000000000000000000000000000000000000000000000000000000);

    superVault_deposit(2);

    property_naivePPSDoesntChangeOnDepositOrMint();
}
```

Mitigation

Yield updates should be vested over some time, this can be enforced offchain by having gradual updates to the PPS.

Additionally minting fees should be set to be higher than the average oracle update deviation to nullify the benefit of frontrunning

M-13 `emergencyPrice` being used in a try catch without enforcing min gas could be triggered by an attacker

Impact

`emergencyPrice` is generally speaking a dangerous addition to the codebase as it can cause a step-wise change from the "real price" to an hardcoded price.

Said price transition is definitely not ideal in most cases.

However this transition becomes very dangerous when an external caller can trigger it

The way `emergencyPrice` is used looks as follows: <https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/libraries/SuperAssetPriceLib.sol#L168-L177>

```
try superOracle.getQuoteFromProvider(one, token, USD, AVERAGE_PROVIDER) returns (
    uint256 _priceUSD, uint256 _stddev, uint256, uint256 _m
) {
    priceUSD = _priceUSD;
    stddev = _stddev;
    M = _m;
} catch {
    priceUSD = superOracle.getEmergencyPrice(token);
    M = 0;
}
```

This is performing a `try/catch` which will capture all revert reasons, including running out gas.

Due to EIP150 an external call (`superOracle.getQuoteFromProvider`) reverting would leave 1/64 gas left for execution. By carefully pre-heating the remaining storage slots an attacker may be able to have sufficient gas for a transaction to go through while using the emergency price.

Mitigation

You can either enforce that there's enough gas before making the call (audited multiple times):

<https://github.com/ethereum-optimism/optimism/blob/94af69bdb0d551f71c7e4c7ee610f1b053f2798a/packages/contracts-bedrock/src/universal/CrossDomainMessenger.sol#L278-L309>

Or you can use this check, used for Liquity Oracles, which has been audited multiple times:

<https://github.com/liquity/bold/blob/3105be782b16b1e36a16b43f75e174df55b81019/contracts/src/PriceFeeds/MainnetL105>

M-14 Malicious Main Manager can prevent from being removed by spam creating secondaryManager

Impact

`changePrimaryManager` is an emergency operation meant to kick a malicious manager and all secondary managers.

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultAggregator.sol#L528-L563>

```
/// @inheritdoc ISuperVaultAggregator
function changePrimaryManager(address strategy, address newManager) external validStrategy(strategy) {
    // Only SuperGovernor can call this
    if (msg.sender != address(SUPER_GOVERNOR)) {
        revert UNAUTHORIZED_UPDATE_AUTHORITY();
    }

    if (strategy == address(0)) revert ZERO_ADDRESS();

    if (newManager == address(0)) revert ZERO_ADDRESS();

    address oldManager = _strategyData[strategy].mainManager;

    // SECURITY: Clear any pending manager proposals to prevent malicious re-takeover
    _strategyData[strategy].proposedManager = address(0);
    _strategyData[strategy].managerChangeEffectiveTime = 0;

    // SECURITY: Clear any pending hooks root proposals to prevent malicious hook updates
    _strategyData[strategy].proposedHooksRoot = bytes32(0);
    _strategyData[strategy].hooksRootEffectiveTime = 0;

    // SECURITY: Clear all secondary managers as they may be controlled by malicious manager
    // Get all secondary managers first to emit proper events
    address[] memory clearedSecondaryManagers = _strategyData[strategy].secondaryManagers.values();

    // Clear the entire secondary managers set
    for (uint256 i = 0; i < clearedSecondaryManagers.length; i++) {
        _strategyData[strategy].secondaryManagers.remove(clearedSecondaryManagers[i]);
        emit SecondaryManagerRemoved(strategy, clearedSecondaryManagers[i]);
    }

    // Set the new primary manager
    _strategyData[strategy].mainManager = newManager;

    emit PrimaryManagerChanged(strategy, oldManager, newManager);
}
```

Because of how it's coded, where it will attempt to remove all secondary managers.

And because there's no limit on how many secondary managers can be added to a strategy.

A malicious primary manager can make `changePrimaryManager` permanently revert by adding a high amount of secondary managers.

This will cause this loop to go OOG: <https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultAggregator.sol#L549-L557>

```
// SECURITY: Clear all secondary managers as they may be controlled by malicious manager
// Get all secondary managers first to emit proper events
address[] memory clearedSecondaryManagers = _strategyData[strategy].secondaryManagers.values();

// Clear the entire secondary managers set
for (uint256 i = 0; i < clearedSecondaryManagers.length; i++) {
    _strategyData[strategy].secondaryManagers.remove(clearedSecondaryManagers[i]);
    emit SecondaryManagerRemoved(strategy, clearedSecondaryManagers[i]);
}
```

POC

- Create new strategy
- Add 2 full blocks of secondary managers
- Try calling `changePrimaryManager`
- It will always revert

Mitigation

Enforce a hard limit on the maximum number of secondary managers (e.g. 100)

Q-01 `Vault.withdraw` `shares` should roundUp

Impact

Whenever a withdraw happens we have to pay shares Rounding this down makes it cheaper than intended
From high level this is QA as it's not tied to the PPS (hardcoded by admin)

Snippet

```
shares = assets.mulDiv(  
    PRECISION,  
    averageWithdrawPrice,  
    Math.Rounding.Floor  
);
```

Mitigation

```
shares = assets.mulDiv(  
    PRECISION,  
    averageWithdrawPrice,  
    Math.Rounding.Ceil  
);
```

Q-02 `previewDeposit` and `previewMint` divergence

When the price is set very low in the `ECDSAPPSOracle` it causes the `previewDeposit` and `previewMint` to diverge.

See [the following reproducer](#) :

```
function test_property_previewEquivalenceFromShares_1() public {
    vm.warp(block.timestamp + 5);

    vm.roll(block.number + 1);

    ECDSAPPSOracle_updatePPS_clamped(1);

    console2.log("precision: ", superVault.PRECISION());
    property_previewEquivalenceFromShares(1);
}
```

The value returned by `previewDeposit` is capped by `superVault.PRECISION()` because `previewMint` returns a value of 2 shares which when the 10% performance fee is applied causes the return value to be capped by `PRECISION` .

Q-03 `manageEmergencyWithdraw` phase 2 is not necessary and can be removed

Impact

`manageEmergencyWithdraw` looks as follows

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L413-L425)

[periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L413-L425](https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L413-L425)

```
function manageEmergencyWithdraw(uint8 action, address recipient, uint256 amount) external {
    if (action == 1) {
        _proposeEmergencyWithdraw();
    } else if (action == 2) {
        _executeEmergencyWithdrawActivation();
    } else if (action == 3) {
        _performEmergencyWithdraw(recipient, amount);
    } else if (action == 4) {
        _cancelEmergencyWithdrawProposal();
    } else {
        revert ACTION_TYPE_DISALLOWED();
    }
}
```

Where `_executeEmergencyWithdrawActivation` looks like this: [https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L980-L988)

[periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L980-L988](https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L980-L988)

```
function _executeEmergencyWithdrawActivation() internal {
    if (emergencyWithdrawableEffectiveTime == 0) revert NO_PROPOSAL();
    if (block.timestamp < emergencyWithdrawableEffectiveTime) revert INVALID_TIMESTAMP();
    emergencyWithdrawable = proposedEmergencyWithdrawable;
    proposedEmergencyWithdrawable = false;
    emergencyWithdrawableEffectiveTime = 0;
    emit EmergencyWithdrawableUpdated(emergencyWithdrawable);
}
```

Performing the withdrawal can only be done by the primary manager

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L1003-L1012)

[periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L1003-L1012](https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L1003-L1012)

```
function _performEmergencyWithdraw(address recipient, uint256 amount) internal {
    _isPrimaryManager(msg.sender);

    if (!emergencyWithdrawable) revert INVALID_EMERGENCY_WITHDRAWAL();
    if (recipient == address(0)) revert ZERO_ADDRESS();
    uint256 freeAssets = _getTokenBalance(address(_asset), address(this));
    if (amount == 0 || amount > freeAssets) revert INSUFFICIENT_FUNDS();
    _safeTokenTransfer(address(_asset), recipient, amount);
    emit EmergencyWithdrawal(recipient, amount);
}
```

therefore step 2 is not necessary. You could instead remove it and allow the admin to directly perform step 3.

Q-04 Non protected keepers can be made to not be paid out upkeep fees by adding them as `authorizedCallers`

Impact

updating PPS has the following check to not pay the upkeep

<https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultAggregator.sol#L238-L245>

```
// Skip if updateAuthority is in the authorized callers list
// These are manager-designated keepers that should be exempt from fees
// NOTE: Protected keepers cannot be added to this list (blocked in addAuthorizedCaller)
/// @dev: cannot underflow; it's checked above already and it skips the entry if that's the case
if (_strategyData[args.strategies[i]].authorizedCallers.contains(args.updateAuthority)) {
    emit AuthorizedCaller(args.strategies[i], args.updateAuthority);
    continue;
}
```

There's a check to prevent governance mandated keepers from not receiving fees:

<https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultAggregator.sol#L460-L470>

```
function addAuthorizedCaller(address strategy, address caller) external validStrategy(strategy) {
    // Either primary or secondary manager can add authorized callers
    if (!isAnyManager(msg.sender, strategy)) revert UNAUTHORIZED_UPDATE_AUTHORITY();

    if (caller == address(0)) revert ZERO_ADDRESS();

    // Prevent managers from adding protected keepers to circumvent fees
    if (SUPER_GOVERNOR.isProtectedKeeper(caller)) {
        revert CANNOT_ADD_PROTECTED_KEEPER();
    }
}
```

However, this means that anyone else is subject to the same risk

Meaning they may not want to perform the PPS update operation due to the risk of not receiving fees

Mitigation

It seems that `authorizedCallers` in general is a design decision that causes gotchas. It's best to always pay the upkeep Except perhaps when the main manager does the price update (since they would pay themselves)

Q-05 `\_isExemptFromUpkeep` is unused

<https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultAggregator.sol#L1151-L1184>

```
function _isExemptFromUpkeep( /// @audit QA: Unused code
    address strategy,
    address updateAuthority,
    uint256 timestamp
)
    internal
    returns (bool)
{
    // Check if upkeep payments are globally disabled in SuperGovernor
    if (!SUPER_GOVERNOR.isUpkeepPaymentsEnabled()) {
        return true;
    }

    // Update is exempt if it is stale
    if (block.timestamp - timestamp > _strategyData[strategy].maxStaleness) {
        emit StaleUpdate(strategy, updateAuthority, timestamp);
        return true;
    }

    // If manager is a superform manager, they're exempt from upkeep fees
    address manager = _strategyData[strategy].mainManager;
    if (SUPER_GOVERNOR.isSuperformManager(manager)) {
        return true;
    }

    // Check if the updateAuthority is in the authorized callers list
    // These are manager-designated keepers that should be exempt from fees
    // NOTE: Protected keepers cannot be added to this list (blocked in addAuthorizedCaller)
    if (_strategyData[strategy].authorizedCallers.contains(updateAuthority)) {
        return true;
    }

    return false;
}
```

If the code is no longer useful you can delete it

Please note that `authorizedCallers` could be used maliciously to prevent keepers from being paid for their work

Q-06 SuperVaultAggregator allows for misleading vault names

Impact

There's no particular validation for `params.asset, params.name, params.symbol`

<https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultAggregator.sol#L137-L146>

```
vars.currentNonce = _vaultCreationNonce++;
vars.salt = keccak256(abi.encode(msg.sender, params.asset, params.name, params.symbol, vars.currentNonce));

// Create minimal proxies
superVault = VAULT_IMPLEMENTATION.cloneDeterministic(vars.salt);
escrow = ESCROW_IMPLEMENTATION.cloneDeterministic(vars.salt);
strategy = STRATEGY_IMPLEMENTATION.cloneDeterministic(vars.salt);

// Initialize superVault
SuperVault(superVault).initialize(params.asset, params.name, params.symbol, strategy, escrow);
```

End users will have to be careful to review the actual asset and symbol rather than rely on the vault description

Q-07 `previewMint` return value with 100% fees

Impact

When the `feeBps` is set to 100bps, the `previewMint` function returns 0:

```
function previewMint(uint256 shares) public view override returns (uint256) {
    uint256 pps = _getStoredPPS();
    if (pps == 0) return 0;

    uint256 assetsNet = Math.mulDiv(shares, pps, PRECISION, Math.Rounding.Ceil);

    (uint256 feeBps,) = _getManagementFeeConfig();
    if (feeBps == 0) return assetsNet;
    /// @audit line below is the issue, should return uint256.max instead
    if (feeBps >= BPS_PRECISION) return 0; // impossible to mint (would require infinite gross)
    ...
}
```

this causes the `property_comparePreviewMintAndConvertToAssets` property to break with the following reproducer in `CriticToFoundry`:

POC

```
function test_property_comparePreviewMintAndConvertToAssets_13() public {
    superVaultStrategy_proposeVaultFeeConfigUpdate(
        0,
        10000,
        0x0000000000000000000000000000000000000000000000000000000000000000DeaDBeef
    );

    vm.warp(block.timestamp + 237093);

    vm.roll(block.number + 1);

    vm.roll(block.number + 1);
    vm.warp(block.timestamp + 367768);
    superVaultStrategy_executeVaultFeeConfigUpdate();

    property_comparePreviewMintAndConvertToAssets(1);
}
```

Indicating that the value returned by `convertToAssets` is inconsistent with the value returned by `previewMint`.

This also causes a similar divergence when comparing the output of `previewMint` with `previewDeposit`:

```
function test_property_previewEquivalenceFromAssets_1() public {
    superVaultStrategy_proposeVaultFeeConfigUpdate(
        0,
        10000,
        0x0000000000000000000000000000000000000000DeaDBeef
    );

    vm.warp(block.timestamp + 577107);

    vm.roll(block.number + 1);

    vm.roll(block.number + 1);
    vm.warp(block.timestamp + 27732);
    // setting fees to 100%
    superVaultStrategy_executeVaultFeeConfigUpdate();

    // previewMint explicitly returns 0 with 100% fees
    property_previewEquivalenceFromAssets(1);
}
```

Mitigation

Per the spec, since Mint will always revert when the fee is 100% previewMint could also be made to revert: <https://eips.ethereum.org/EIPS/eip->

4626#:~:text=MUST%20NOT%20revert%20due%20to%20vault%20specific%20user/global%20limits.%20MAY%20revert%20d

Q-08 Small Precision Loss in `SuperOracleBase` due to separating the computation to have two divisions

Impact

Separating the code into two divisions causes additional precision loss

<https://github.com/superform-xyz/v2-periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/SuperOracleBase.sol#L340-L350>

```
try AggregatorV3Interface(oracle).decimals() returns (uint8 feedDecimals) {
    uint8 baseDecimals = IERC20(base).safeDecimals();
    uint8 quoteDecimals = IERC20(quote).safeDecimals();

    // Calculate quote amount with proper decimal scaling
    quoteAmount = Math.mulDiv(baseAmount, uint256(answer), 10 ** feedDecimals); /// @audit Early truncation =
Precision loss
    quoteAmount = Math.mulDiv(quoteAmount, 10 ** quoteDecimals, 10 ** baseDecimals);
} catch {
    if (revertOnError) revert ORACLE_DECIMALS_CALL_FAIL(oracle);
    return 0;
}
```

Q-09 Fulfillment reverts if funds haven't been deposited into yield strategy

The `fulfillRedeemRequests` function requires that hooks are passed to it to withdraw from a given yield strategy.

This causes any redemptions requested before the funds are deposited into a yield strategy to revert even if the strategy holds sufficient funds for the redemption, as demonstrated by this test in the

`CriticToFoundry` contract:

```
function test_superVaultStrategy_fulfillRedeemRequests_no_investment()
    public
{
    // Setup yield source
    superVaultStrategy_manageYieldSource_clamped(
        uint256(YieldSourceType.ERC4626)
    );

    // User deposits into SuperVault to create shares
    switchActor(1);
    address user1 = _getActor();
    superVault_deposit(1000e18);

    // Manager invests funds into yield source using executeHooks
    switchActor(0);

    // Multiple users request redemptions with the same amount
    switchActor(1);
    uint256 redeemAmt = 100e18;
    superVault_requestRedeem(redeemAmt);

    // Second user also requests the same amount
    switchActor(2);
    address user2 = _getActor();
    superVault_deposit(500e18);
    superVault_requestRedeem(redeemAmt);

    superVaultStrategy_fulfillRedeemRequests_clamped(redeemAmt);

    // Check that at least one user has their withdraw price set
    _verifyRedemptionFulfillment(user1, user2, redeemAmt);
}
```

This is relatively inefficient as it requires funds to first be invested then withdrawn just to process a redemption even if the strategy already has sufficient balance for the fulfillment.

Q-10 Collection of QA items for `SuperBank`

Should round up

Rounding up ensures the fee is always paid, QA at most

```
// Calculate amounts for sUP and Treasury
uint256 supAmount = upAmount.mulDiv(revenueShare, BPS_MAX);
```

Small Gotcha Race condition if you change the fee

If the fee value is changed before calling `distribute` then you can have some "unfair" / unintended distribution.

This is just a small gotcha

```
address treasury = SUPER_GOVERNOR.getAddress(SUPER_GOVERNOR.TREASURY());

// Get revenue share percentage from SuperGovernor
uint256 revenueShare = SUPER_GOVERNOR.getFee(FeeType.REVENUE_SHARE);
```

QA: This could be done earlier to make the revert case cheaper

Doing the check after: `address upToken = SUPER_GOVERNOR.getAddress(SUPER_GOVERNOR.UP());`

Makes it fail earlier

Just a small gas improvement on the revert case

```
if (up.balanceOf(address(this)) < upAmount) revert INVALID_UP_AMOUNT_TO_DISTRIBUTE();
```

Q-11 Collection of QA items for `ECDSAPPSOracle`

Gas: This can be verified earlier

```
if (validatorSet != proofsLength) revert INVALID_VALIDATOR_SET();
```

QA: Race conditions here based on when governance adds and removes validators and their count

```
uint256 actualTotalValidators = SUPER_GOVERNOR.getValidators().length;  
if (totalValidators != actualTotalValidators) revert INVALID_TOTAL_VALIDATORS();
```

Gas: When doing batch validation, you can:

- Perform operations that require no calls earlier
- Create a cache of validators and use the cache to save gas (skip calls)
- Cache the result for totalValidators since it can be reused

```
_validateProofs(  
    args.strategies[i],  
    args.proofsArray[i],  
    args.ppss[i],  
    args.ppsStdevs[i],  
    args.validatorSets[i],  
    args.totalValidators[i],  
    args.timestamps[i]  
);
```

Gas: This is done in the aggregator already (could comment it with a note, or keep)

But for batch this is being repeated n times, which is not necessary

```
if (!SUPER_GOVERNOR.isActivePPSOracle(address(this))) revert NOT_ACTIVE_PPS_ORACLE();
```

Q-12 Collection of QA items for `SuperGovernor`

GAS: Can use the constant value: 4000

```
_feeValues[FeeType.SUPER_ASSET_SWAP_FEE] = 4000; // 40% swap fee
emit FeeUpdated(FeeType.REVENUE_SHARE, _feeValues[FeeType.REVENUE_SHARE]); /// @audit Reading from storage is more expensive
```

QA: Common Contract Keys have a low likelihood of clash

```
bytes32 public constant UP = keccak256("UP");
```

If you use something like chainID | address or even the UTF-8 string converted to bytes - there can be no clash

QA: Setters don't emit the old value

```
function setAddress(bytes32 key, address value) external onlyRole(_SUPER_GOVERNOR_ROLE) {
    if (value == address(0)) revert INVALID_ADDRESS();

    _addressRegistry[key] = value;
    emit AddressSet(key, value);
}

/*//////////////////////////////////////
    PERIPHERY CONFIGURATIONS
//////////////////////////////////////*/
/// @inheritdoc ISuperGovernor
function setProver(address prover) external onlyRole(_SUPER_GOVERNOR_ROLE) {
    if (prover == address(0)) revert INVALID_ADDRESS();

    _prover = prover;
    emit ProverSet(prover);
}
```

QA: Typo in _minStalenesEffectiveTime -> _minStalenessEffectiveTime

Q-13 You can `withdraw` some assets and "burn" 0 shares due to incorrect rounding direction

Impact

`withdraw` is rounding down the `shares` value

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVault.sol#L398-L424>

```
function withdraw(
    uint256 assets,
    address receiver,
    address controller
)
    public
    override
    nonReentrant
    returns (uint256 shares)
{
    if (receiver == address(0)) revert ZERO_ADDRESS();
    _validateController(controller);

    uint256 averageWithdrawPrice = strategy.getAverageWithdrawPrice(controller);
    if (averageWithdrawPrice == 0) revert INVALID_WITHDRAW_PRICE();

    uint256 maxWithdrawAmount = maxWithdraw(controller);
    if (assets > maxWithdrawAmount) revert INVALID_AMOUNT();

    // Calculate shares based on assets and average withdraw price
    shares = assets.mulDiv(PRECISION, averageWithdrawPrice, Math.Rounding.Floor); /// @audit Rounding down!

    // Take assets from strategy (7540 path)
    strategy.handleOperations7540(ISuperVaultStrategy.Operation.ClaimRedeem, controller, receiver, assets);

    emit Withdraw(msg.sender, receiver, controller, assets, shares);
}
```

This is inconsistent with how ERC4626 vaults work.

Since the value is only used in an event this should cause no particular risk

However the event `emit Withdraw(msg.sender, receiver, controller, assets, shares)` will emit 0 shares being claimed, while some assets will be moved

Q-14 `\_processRedeemFulfillments` can be grieved via front-running

Impact

The code in `_processRedeemFulfillments`

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L680-L688>

```
uint256 totalRequestedAmount = 0;
uint256[] memory controllerRequestedAmount = new uint256[](controllersLength);
for (uint256 i; i < controllersLength; ++i) {
    controllerRequestedAmount[i] = superVaultState[controllers[i]].pendingRedeemRequest;
    totalRequestedAmount += controllerRequestedAmount[i];
} /// @audit TODO: Controller can front-run and grief this
if (processedShares + TOLERANCE_CONSTANT < totalRequestedAmount) {
    revert INVALID_REDEEM_FILL();
}
```

Means that for each controller, the manager has to redeem 100% of their shares, minus a small constant.

This means that any controller can grief the processing of redemptions by asking to redeem a few more shares, or by cancelling and performing a new request with a slightly lower amount.

Mitigation

I believe this should be accepted as is, with the possibility of delaying / refusing to redeem griefers.

Adding partial redemptions could be good for other reasons, but must be done only once the redemption mechanism is cognizant of losses during the withdrawal of underlying assets

Q-15 `\_getSlippageTolerance` is generally not necessary

Impact

Many parts of the code have snippets such as:

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L658-L660>

```
if (vars.outAmount == 0) revert ZERO_OUTPUT_AMOUNT();
if (vars.outAmount * BPS_PRECISION < expectedAssetOutput * (BPS_PRECISION - _getSlippageTolerance())) {
    revert MINIMUM_OUTPUT_AMOUNT_ASSETS_NOT_MET();
}
```

Where they take a resulting token amount and compare it against the manager provided expected amount out, which is then reduced by `_getSlippageTolerance`

This can be refactored to:

```
vars.outAmount < expectedAssetOutput {
    revert MINIMUM_OUTPUT_AMOUNT_ASSETS_NOT_MET();
}
```

Where `expectedAssetOutput` is already set to a value that includes the slippage

`expectedAssetOutput` could already include some slippage, this makes the code simpler with no particular additional gotcha

Q-16 `distribute` should roundUp `supAmount`

Impact

The logic for `distribute` is as follows:

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperBank.sol#L56-L59)

[periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperBank.sol#L56-L59](https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperBank.sol#L56-L59)

```
// Calculate amounts for sUP and Treasury
uint256 supAmount = upAmount.mulDiv(revenueShare, BPS_MAX); /// @audit Rounds down

uint256 treasuryAmount = upAmount - supAmount;
```

Since `treasuryAmount` will receive whatever is left, it makes sense to roundUp `supAmount`

Q-17 The `SuperBank` may not be able to receive funds from contracts that transfer funds with some calldata

Impact

The SuperBank has a `receive` function, but no `fallback` function

Contracts that forward some data (e.g. `bank.call(" ")`), won't be able to transfer funds

I'm ambivalent on fixing this issue because on one end it may cause incompatibility with some niche project. On the other hand it could create issues to integrators as they may not revert when calling a non existing function

Q-18 Incorrect extra space in `UPDATE\_PPS\_TYPEHASH`

Impact

UPDATE_PPS_TYPEHASH looks like this:

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/ECDSAPPSOracle.sol#L29-L31)

[periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/ECDSAPPSOracle.sol#L29-L31](https://github.com/superform-xyz/v2-periphery/blob/90db6a2ec6ff0e17f572bdbc29bbca6b910b4733/src/oracles/ECDSAPPSOracle.sol#L29-L31)

```
bytes32 public constant UPDATE_PPS_TYPEHASH = keccak256(
    "UpdatePPS(address strategy,uint256 pps,uint256 ppsStdev,uint256 validatorSet,uint256 totalValidators,uint256
timestamp, uint256 strategyNonce)"
);
```

There's one extra space for the nonce parameter , uint256 nonce)

There's supposed to be no space after the comma

Q-19 Hook.call is still copying the returnData into memory

Impact

The code for `Bank` calling a hook while ignoring it's return value looks as follows:

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/Bank.sol#L88-L89)

[periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/Bank.sol#L88-L89](https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/Bank.sol#L88-L89)

```
(success,) = executionStep.target.call{ value: executionStep.value }(executionStep.callData);
```

The `bytes memory data` is purposefully left out as it's ignored

However, the solidity compiler will add the code to load this data into memory anyway, increasing gas cost.

POC

```
Ran 1 test for test/KnotMemory.t.sol:KnotMemory
[PASS] test_stillGoes00G() (gas: 85824)
Logs:
  returnSize 20000
  initialFMP 128
  finalFMP 20224
```

```

// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.26;

import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";

// forge test --match-contract KnotMemory -vv

contract ReturnBomber {
    function test() public {
        uint256 _bytes = 2_0_000;
        assembly {
            return(0, _bytes)
        }
    }
}

contract ReturnBomb is Test {

    function test_stillGoesOOG() public {
        ReturnBomber returnbomber = new ReturnBomber();

        uint256 initialFMP;
        assembly {
            initialFMP := mload(0x40)
        }

        (bool success,) = address(returnbomber).call{gas: 1e6}(abi.encodeWithSelector(returnbomber.test.selector));

        uint256 finalFMP;
        assembly {
            finalFMP := mload(0x40)
        }

        uint256 returnSize;
        assembly {
            returnSize := returndatasize()
        }

        console.log("returnSize", returnSize);
        console.log("initialFMP", initialFMP);
        console.log("finalFMP", finalFMP);
    }
}

```

Mitigation

You can use this code, originally written by the OP team, then adapted by me for Liquity, audited multiple times.

To actually ignore the return data for the call:

<https://github.com/liquity/V2-gov/blob/b880481d701d2b07236f2e04842c79b4fe77e5fd/src/Utils/SafeCallMinGas.sol#L27-L37>

Q-20 Inconsistent Merkle Roots generation for `Bank` hooks

Impact

As explained to me the way hook's Merkle Roots are generated is by creating proofs that can be validated by inspecting relevant parameters passed to the hook (which can be viewed via `inspect`)

In the case of Bank no parameter is validated:

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/Bank.sol#L77-L85>

```
if (executionStep.target != hookAddress) {
    // Verify that this target is allowed for this hook using the Merkle proof
    // The leaf is the hash of the target address
    targetLeaf = keccak256(bytes.concat(keccak256(abi.encodePacked(executionStep.target))));
    // Verify this target is allowed using the corresponding Merkle proof
    if (!MerkleProof.verify(merkleProof, merkleRoot, targetLeaf)) {
        revert INVALID_MERKLE_PROOF();
    }
}
```

This means that the Leafs in the Merkle Tree will have to be hashed addresses instead of the combination of the addresses and some parameter.

Mitigation

Given the trusted role for the `BankManager` this seems to be just an issue with the documentation

Q-21 `slashStake` not callable

The `SuperVaultAggregator::slashStake` function appears to not be callable because it expects the caller to be the `SUPER_GOVERNOR` contract:

A placeholder for an image, likely a screenshot of a code snippet or a diagram illustrating the function's requirements.

But the `SuperGovernor` implementation doesn't include a function that calls `slashStake`. It appears that the function is meant to be called by the address with the `SUPER_GOVERNOR_ROLE` instead:

A placeholder for an image, likely a screenshot of a code snippet or a diagram illustrating the function's requirements.

This unit tests shows that it currently works with pranking as the `superGovernor` but wouldn't work outside of the test environment:

A placeholder for an image, likely a screenshot of a code snippet or a diagram illustrating the function's requirements.

I-01 Return value of `maxRedeem` is incorrect

The following reproducer demonstrates that the logic for calculating `maxRedeem` is incorrect:

```
function test_cryptic_erc7540_4_redeem_1() public {
    yieldSource_mint(1, 0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    superVaultStrategy_manageYieldSource_clamped(0);

    superVault_deposit(3);

    vm.roll(block.number + 1);
    vm.warp(block.timestamp + 5);
    superVault_requestRedeem_clamped(2);

    console2.log(
        "PPS before: %e",
        superVaultAggregator.getPPS(address(superVaultStrategy))
    );
    ECDSAPPSOracle_updatePPS_clamped(950051690458586526);
    console2.log(
        "PPS after: %e",
        superVaultAggregator.getPPS(address(superVaultStrategy))
    );

    console2.log(
        "avg withdraw price before fulfill: %e",
        superVaultStrategy.getAverageWithdrawPrice(_getActor())
    );
    superVaultStrategy_fulfillRedeemRequests_clamped(2);

    console2.log(
        "avg withdraw price after fulfill: %e",
        superVaultStrategy.getAverageWithdrawPrice(_getActor())
    );
    cryptic_erc7540_4_redeem(1);
}
```

this makes the following property break incorrectly because it appears that the user has redeemed more than their `maxRedeem` :

```
function erc7540_4_redeem(
    address erc7540Target,
    uint256 amt
) public virtual returns (bool) {
    // Skip 0
    if (amt == 0) {
        return true;
    }

    uint256 maxDep = IERC7540Like(erc7540Target).maxRedeem(actor);

    uint256 sum = maxDep + amt;
    if (sum == 0) {
        return true; // Needs to be greater than 0
    }

    try IERC7540Like(erc7540Target).redeem(maxDep + amt, actor, actor) {
        return false;
    } catch {
        // We want this to be hit
        return true; // So we explicitly return here, as a means to ensure that this is the code path
    }

    // NOTE: This code path is never hit per the above
}
```

however the check on the following lines of `SuperVault::redeem` ensure the user doesn't redeem more assets than their `maxWithdraw`:

```
function redeem(
    uint256 shares,
    address receiver,
    address controller
) public override nonReentrant returns (uint256 assets) {
    ...

    // Calculate assets based on shares and average withdraw price
    assets = shares.mulDiv(
        averageWithdrawPrice,
        PRECISION,
        Math.Rounding.Floor
    );

    uint256 maxWithdrawAmount = maxWithdraw(controller);
    console2.log("assets: ", assets);
    console2.log("maxWithdrawAmount: ", maxWithdrawAmount);
    if (assets > maxWithdrawAmount) revert INVALID_AMOUNT();
    ....
}
```

Logs:

```
assets: 1
maxWithdrawAmount: 1
```

In the `redeem` function, the `maxWithdraw` is the limiting factor, but in the `maxRedeem` function the `maxWithdraw` is used to calculate the redeemable share amount:

```
function maxRedeem(address owner) public view override returns (uint256) {
    uint256 withdrawPrice = strategy.getAverageWithdrawPrice(owner);
    if (withdrawPrice == 0) return 0;
    return
        maxWithdraw(owner).mulDiv(
            PRECISION,
            withdrawPrice,
            Math.Rounding.Floor
        );
}
```

I-02 Invariant Testing Suggested Next Steps

Summary

Over the course of 4 weeks an invariant testing suite was built to test 51 properties. The implemented properties are outlined in the [properties-table](#) file.

After running Echidna for 100 million+ runs the following issues were discovered.

The latest run is available [here](#).

Findings

1. `previewMint` return value with 100% fees
2. `previewDeposit` and `previewMint` divergence
3. Donations to yield source cause a loss on withdrawal for users
4. `fulfillRedeemRequests` can cause `accumulatorShares` and `accumulatorCostBasis` to not decrease correctly
5. Attempting to withdraw `maxWithdraw` reverts
6. Insolvency in `SuperVaultStrategy`
7. Unbacked shares remain on withdrawal
8. Users can lose value on mint/redeem
9. Return value of `maxRedeem` is incorrect

Recommendations

1. Mocking yield sources that use swapping and rewards would provide more realistic testing as the current yield source mocks only mock 4626, 7540 and 5115 vaults. Given that swapping and rewards yield sources only expect to be interacted with via `executeHooks` this may cause inconsistencies if there's logic that normally gets executed in a redemption fulfillment that doesn't get executed via `executeHooks`.
2. An optimization test addressing the issue outlined [here](#) would be valuable in helping to determine the maximum possible impact of rounding down in the cost basis ratio calculation. From manual review and modeling in excel it seems like the impact is limited but an optimization test would give greater insight, the primary difficulty is in determining how to implement the optimization.
3. Checking `redeem` / `withdraw` never revert due to underflow. In the currently handler setup these can easily revert due to overflow with large share values and a high price. This would therefore be best tested with a stateless fuzz test to determine that it doesn't revert due to underflow for valid values.
4. Identifying the source of `maxWithdraw` issue [here](#) would allow determining if this poses real risk of loss of funds or is only an inconsistent calculation issue.

Admin Mistakes

The following identify issues that an admin can make that lead to breaking system properties:

1. After a redemption is fulfilled and the funds are transferred to the `SuperVaultStrategy` from the yield strategy in which they were invested in an admin can prevent users calling `withdraw` / `redeem` by reinvesting the funds via `executeHooks`. This leaves users with a claimable amount but no way to claim it unless the admin calls `executeHooks` again to transfer funds back to the `SuperVaultStrategy`. For this reason the `_claimableMoreThanInvested` function checks the amount `superVaultStrategy_executeHooks_clamped` can deposit into a yield strategy, otherwise a significant number of properties break.
2. If an admin fulfills a redemption at a high price it can lock in insolvency. See the issue [here](#)

The following properties would be good candidates for live monitoring to ensure that the above admin mistakes can't cause issues in production:

- doomsday_allUsersCanRedeem
- property_sumOfClaimable
- property_superVaultStrategySolvency
- crytic_erc7540_7_withdraw
- crytic_erc7540_7_redeem
- property_assetBacking
- doomsday_mintRedeemSymmetrical

I-03 Users can lose value on mint/redeem

When the oracle PPS is set very high and the yield source experiences a gain it can cause the SuperVault shares to decrease in value such that a user can lose assets just by minting and redeeming.

See the [following reproducer](#):

```
function test_doomsday_mintRedeemSymmetrical_5() public {
    superVaultStrategy_manageYieldSource_clamped(0);

    yieldSource_mint(1, 0x0000000000000000000000000000000000000000);

    vm.roll(block.number + 1);
    vm.warp(block.timestamp + 5);
    ECDSAPPSOracle_updatePPS_clamped(
        45875423970713493951589436881765514280565129916122376120788407117094766
    );

    yieldSource_simulateGain(157404);

    doomsday_mintRedeemSymmetrical(2);
}
```

Seems like the source is similar to the issue outlined [here](#)

I-04 Invariant Testing Result

Summary

Over the course of 4 weeks an invariant testing suite was built to test 51 properties. The implemented properties are outlined in the [properties-table](#) file.

After running Echidna for 100 million+ runs the following issues were discovered.

Findings

1. `previewMint` return value with 100% fees
2. `previewDeposit` and `previewMint` divergence
3. Donations to yield source cause a loss on withdrawal for users
4. `fulfillRedeemRequests` can cause `accumulatorShares` and `accumulatorCostBasis` to not decrease correctly
5. Attempting to withdraw `maxWithdraw` reverts
6. Insolvency in `SuperVaultStrategy`
7. Unbacked shares remain on withdrawal
8. Users can lose value on mint/redeem

Recommendations

1. Mocking yield sources that use swapping and rewards would provide more realistic testing as the current yield source mocks only mock 4626, 7540 and 5115 vaults. Given that swapping and rewards yield sources only expect to be interacted with via `executeHooks` this may cause inconsistencies if there's logic that normally gets executed in a redemption fulfillment that doesn't get executed via `executeHooks`.
2. An optimization test addressing the issue outlined [here](#) would be valuable in helping to determine the maximum possible impact of rounding down in the cost basis ratio calculation. From manual review and modeling in excel it seems like the impact is limited but an optimization test would give greater insight, the primary difficulty is in determining how to implement the optimization.
3. Checking `redeem / withdraw` never revert due to underflow. In the currently handler setup these can easily revert due to overflow with large share values and a high price. This would therefore be best tested with a stateless fuzz test to determine that it doesn't revert due to underflow for valid values.
4. Identifying the source of `maxWithdraw` issue [here](#) would allow determining if this poses real risk of loss of funds or is only an inconsistent calculation issue.

Admin Mistakes

The following identify issues that an admin can make that lead to breaking system properties:

1. After a redemption is fulfilled and the funds are transferred to the `SuperVaultStrategy` from the yield strategy in which they were invested in an admin can prevent users calling `withdraw / redeem` by reinvesting the funds via `executeHooks`. This leaves users with a claimable amount but no way to claim it unless the admin calls `executeHooks` again to transfer funds back to the `SuperVaultStrategy`. For this reason the `_claimableMoreThanInvested` function checks the amount `superVaultStrategy_executeHooks_clamped` can deposit into a yield strategy, otherwise a significant number of properties break.
2. If an admin fulfills a redemption at a high price it can lock in insolvency. See the issue [here](#)

The following properties would be good candidates for live monitoring to ensure that the above admin mistakes can't cause issues in production:

- `doomsday_allUsersCanRedeem`
- `property_sumOfClaimable`
- `property_superVaultStrategySolveny`

- `crytic_erc7540_7_withdraw`
- `crytic_erc7540_7_redeem`
- `property_assetBacking`
- `doomsday_mintRedeemSymmetrical`

I-05 Unbacked shares remain on withdrawal when withdrawals cause insolvency

The following reproducer shows that withdrawals can leave unbacked shares:

```
function test_property_assetBacking_10() public {
    yieldSource_mint(1, 0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    superVaultStrategy_manageYieldSource_clamped(0);

    yieldSource_mint(1, 0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    superVault_deposit(4);

    superVault_deposit(4);

    superVault_requestRedeem_clamped(5);

    vm.warp(block.timestamp + 5);

    vm.roll(block.number + 1);

    ECDSAPPSOracle_updatePPS_clamped(
        115792089237316195423570985008687907853269984665640564039457584007913129639931
    );

    yieldSource_simulateGain(100000003);

    superVaultStrategy_fulfillRedeemRequests_clamped(4);

    superVault_withdraw(87234118);

    // have 2 unbacked shares
    console2.log("totalSupply: ", superVault.totalSupply());
    property_assetBacking();
}
```

This reproducer only shows 2 unbacked shares so I've implemented the following [optimization test](#) to optimize the amount of unbacked `SuperVault` shares:

```
function optimize_assetBackingDifference() public view returns (int256) {
    uint256 summedTotalAssets = _sumStrategyAssets();

    if (summedTotalAssets == 0) {
        return int256(superVault.totalSupply());
    }
}
```

And started a job here to determine if the unbacked shares is ever > 2:

<https://getrecon.xyz/shares/6ce39907-0142-4d5d-a84a-b463ffa77279>

I-06 Insolvency in `SuperVaultStrategy`

The [following reproducer](#) demonstrates that it's possible for the strategy balance to be less than the sum of all claimable assets calculated via `maxWithdraw` in the `SuperVaultStrategy`:

```
function test_property_sumOfClaimable_5() public {
    yieldSource_mint(1, 0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    superVaultStrategy_manageYieldSource_clamped(0);

    yieldSource_mint(1, 0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    superVault_deposit(3);

    superVault_requestRedeem_clamped(1);

    vm.warp(block.timestamp + 5);

    vm.roll(block.number + 1);

    ECDSAPPSOracle_updatePPS_clamped(
        700960855099362077226925743595804258294593977845093495232344554
    );

    yieldSource_simulateGain(973782);

    superVaultStrategy_fulfillRedeemRequests_clamped(1);

    superVault_requestRedeem_clamped(1);

    superVaultStrategy_fulfillRedeemRequests_clamped(1);

    property_sumOfClaimable();
}
```

Failing tests:

Encountered 1 failing test in test/recon/CryticToFoundry.sol:CryticToFoundry

[FAIL: sum of all claimable is > balance available: 1168720 > 779146] test_property_sumOfClaimable_5() (gas: 2807567)

I-07 Attempting to withdraw `maxWithdraw` reverts

The [following reproducer](#) demonstrates that it's possible for `maxWithdraw` to return a value greater than the `SuperVaultStrategy`'s current balance and cause the withdrawal to revert due to insufficient balance:

```
function test_doomsday_maxWithdrawResetsAfterFullWithdrawal_17() public {
    yieldSource_mint(1, 0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    superVaultStrategy_manageYieldSource_clamped(0);

    yieldSource_mint(1, 0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    superVault_deposit(3);

    superVault_requestRedeem_clamped(1);

    vm.warp(block.timestamp + 5);

    vm.roll(block.number + 1);

    ECDSAPPSOracle_updatePPS_clamped(
        700960855099362077226925743595804258294593977845093495232344554
    );

    yieldSource_simulateGain(973782);

    superVaultStrategy_fulfillRedeemRequests_clamped(1);

    superVault_requestRedeem_clamped(1);

    doomsday_maxWithdrawResetsAfterFullWithdrawal(988620);
}
```

the issue is caused by the check in [this line](#):

```
if (
    assetsToClaim > remainingAssets &&
    assetsToClaim - remainingAssets <= TOLERANCE_CONSTANT
) {
    actualAmountToClaim = remainingAssets;
}
```

which is essentially bypassed, allowing the `assetsToClaim > remainingAssets` :

```
Logs:
actualAmountToClaim before: 2148006
assetsToClaim > remainingAssets: true
assetsToClaim - remainingAssets <= TOLERANCE_CONSTANT false
actualAmountToClaim after: 2148006
state.maxWithdraw < actualAmountToClaim: false
state.maxWithdraw: 2148006
actualAmountToClaim: 2148006
```

which causes the withdrawal to revert due to insufficient balance:



TODO: identify if issue is in the `maxWithdraw` calculation on fulfillment.

- my intuition is that calculation of `maxWithdraw` may be the source of the issue as it's not returning an accurate value but also for consistency the withdrawal should always succeed so it seems like the check in `_handleClaimRedeem` should be stricter

I-08 `fulfillRedeemRequests` can cause `accumulatorShares` to not decrease correctly

The following reproducer shows that `accumulatorShares` decreases by 1 wei more than expected on a call to `fulfillRedeemRequests` :

```
// forge test --match-test test_property_accumulatorSharesDecreaseOnFulfill_exact_6 -vvv
function test_property_accumulatorSharesDecreaseOnFulfill_exact_6() public {

    superVaultStrategy_manageYieldSource_clamped(0);

    yieldSource_mint(1,0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    superVault_deposit(4);

    superVault_requestRedeem_clamped(2);

    superVaultStrategy_fulfillRedeemRequests_clamped(1);

    property_accumulatorSharesDecreaseOnFulfill_exact();

}
```

Note: implemented an additional check in `property_accumulatorSharesDecreaseOnFulfill_with_tolerance` to confirm if the difference is ever greater than 10 wei.

Started a run for this here: <https://getrecon.xyz/shares/1f7a118f-386e-403c-bd5e-ca75a35c24c1>

I-09 Donations to yield source cause a loss on withdrawal for users

The [following reproducers](#) demonstrates that when there's a donation to the underlying yield source vault it causes the user to lose 1 wei worth of assets:

NOTE: a `TOLERANCE` value was added to `doomsday_mintRedeemSymmetrical` so for this reproducer to work as expected just remove the tolerance from the `doomsday_mintRedeemSymmetrical` function.

```
// forge test --match-test test_doomsday_mintRedeemSymmetrical_4 -vvv
// NOTE: requires admin hasn't called executeHooks to deposit into strategy yet but still fulfills
function test_doomsday_mintRedeemSymmetrical_4() public {
    superVaultStrategy_manageYieldSource_clamped(0);

    switchActor(1);
    yieldSource_mint(5, 0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    doomsday_mintRedeemSymmetrical(1);
}
```

similarly:

```
// forge test --match-test test_doomsday_depositWithdrawSymmetrical_5 -vvv
function test_doomsday_depositWithdrawSymmetrical_5() public {
    superVaultStrategy_manageYieldSource_clamped(0);

    yieldSource_mint(1, 0xc3C1658B1e3b9e017030807d0C50895456FD2379);

    switchActor(1);

    (
        uint256 balanceAfter,
        uint256 balanceBefore
    ) = doomsday_depositWithdrawSymmetrical(2);
}
```

I-10 Remainder optimization tests

Preamble

Moving shares accumulators has an inherent rounding error:

[https://github.com/superform-xyz/v2-](https://github.com/superform-xyz/v2-periphery/blob/155ee6d4f7e39d1fcf2f019b78673188785f8466/src/SuperVault/SuperVaultStrategy.sol#L532-L540)

[periphery/blob/155ee6d4f7e39d1fcf2f019b78673188785f8466/src/SuperVault/SuperVaultStrategy.sol#L532-L540](https://github.com/superform-xyz/v2-periphery/blob/155ee6d4f7e39d1fcf2f019b78673188785f8466/src/SuperVault/SuperVaultStrategy.sol#L532-L540)

```
// Pro-rata move of cost basis (NO PPS here; preserves fee correctness)
uint256 movedCostBasis = sharesToMove == availableAccumulatorShares
    ? fromState.accumulatorCostBasis
    : Math.mulDiv(
        sharesToMove,
        fromState.accumulatorCostBasis,
        availableAccumulatorShares,
        Math.Rounding.Floor
    );
```

I'm trying to escalate this to weaponize it, the goal being to perform small transfers of tokens

Current conclusion

Rounding can be used to distribute the remainder (spread over multiples in the mantissa) as to effectively transfer a lower ratio (higher fees) to the recipient, this makes the ratio of Basis/Shares move upwards to the next integer, once it does, rounding will no longer be relevant (when Basis/Shares reaches a new integer all remainders are 0).

This leads me to conclude that the maximum impact is at most gaining 1 wei of basis (paying 1 wei less of fees) and sending 1 wei more of fees to the recipient.

Which is a very low impact.

Math Intuition

Assume the costBasis / Shares is 3.25

Rounding will cause 3 to be moved Moving 3 means that the .25 is kept

We reduce the basis of the caller by 3 and add 3 to the basis of the recipient We should instead add 3.25

This causes the basis / shares of the sender to increase

As we repeat this the ratio of basis/shares for the sender increases to 4

Once it hits 4, we have no rounding, so we can no longer weaponize it

Chat with Claude (math can have issue)

<https://claude.ai/share/8e51adc5-bde3-4520-b036-3d1a8801bf14>

I-11 Interesting Preconditions

- Can we deflate share value from a default starting state (deposit 1 asset get 2 shares?)

Off of shares being deflated we can increase our own cost basis (meaning we would pay a lower performance fee), we can forward small amounts of assets to other accounts with the goal of using the fee truncation to also not pay on those accounts

```
function test_move_accumulators(
) public {
    uint256 fromState_accumulatorCostBasis = 5;
    uint256 fromState_accumulatorShares = 2;

    uint256 toState_accumulatorCostBasis = 0;
    uint256 toState_accumulatorShares = 0;

    uint256 sharesToMove = 1;
    uint256 availableAccumulatorShares = fromState_accumulatorShares;

    console.log("fromState_accumulatorShares B4", fromState_accumulatorShares);
    console.log("fromState_accumulatorCostBasis B4", fromState_accumulatorCostBasis);
    console.log("toState_accumulatorShares B4", toState_accumulatorShares);
    console.log("toState_accumulatorCostBasis B4", toState_accumulatorCostBasis);

    // Pro-rata move of cost basis (NO PPS here; preserves fee correctness)
    uint256 movedCostBasis = sharesToMove == availableAccumulatorShares
        ? fromState_accumulatorCostBasis
        : mulDiv(sharesToMove, fromState_accumulatorCostBasis, availableAccumulatorShares); // Floor by default

    fromState_accumulatorShares -= sharesToMove;
    fromState_accumulatorCostBasis -= movedCostBasis;

    toState_accumulatorShares += sharesToMove;
    toState_accumulatorCostBasis += movedCostBasis;

    console.log("fromState_accumulatorShares", fromState_accumulatorShares);
    console.log("fromState_accumulatorCostBasis", fromState_accumulatorCostBasis);
    console.log("toState_accumulatorShares", toState_accumulatorShares);
    console.log("toState_accumulatorCostBasis", toState_accumulatorCostBasis);
}
```

[PASS] test_move_accumulators() (gas: 16014)

Logs:

```
fromState_accumulatorShares B4 2
fromState_accumulatorCostBasis B4 5
toState_accumulatorShares B4 0
toState_accumulatorCostBasis B4 0
fromState_accumulatorShares 1
fromState_accumulatorCostBasis 3
toState_accumulatorShares 1
toState_accumulatorCostBasis 2
```

$$5/2 = 2.5 \quad 3/1 = 3 \quad 2/1 = 2$$

A-01 Tokens for which a insolvency tollerance may be necessary

In many parts of the code, the constant `TOLERANCE_CONSTANT` is used.

This should never be necessary when it comes to shares, as all math should be exact and since losses are imputed to controllers, no insolvency should happen.

The code is incompatible with FoT tokens and those should never be used, the `TOLERANCE_CONSTANT` would be insufficient to protect against them and would lead to insolvencies. As such FoT tokens should never be used.

There are 2 types of tokens for which the `TOLERANCE_CONSTANT` could be necessary:

- stETH
- aTokens (or similar) Scaled tokens

stETH Edge Case

Description: <https://github.com/lidofinance/core/issues/442> <https://docs.lido.fi/guides/lido-tokens-integration-guide/#1-2-wei-corner-case>

Examples: <https://github.com/sherlock-audit/2025-04-burve-judging/issues/53>

Repros: You can repro this by using a Mock StETH Token as part of the assets used in Invariant Testing

aTokens Edge Case We have investigated this edge case when working with Cap, for them it applies to their debt token so this would have no impact. However their code is very similar to AAVes and I believe this finding would apply to you if you used aTokens as the `asset`

Whenever the token is moved, an underlying index is used to compute the shares that should be moved, this causes a rounding error of up to 1 wei (assuming the token has liquidity)

Repros: You can repro this by using the aToken as asset

Mitigation

Don't use tokens with underlying shares such as stETH or aTokens as assets

If you chose to use them, monitor the solvency of the contract and have the manager be willing to perform a small (1 to 10 wei) donation to the escrow.

A-02 Incomplete Notes on accounting/oracles

Executive Summary

The oracles were not in scope for this review

I gave them a quick check

Here's a few notes

Pendle / Spectra

Converting asset to PT token may cause a loss to the Vault if the PT token is discounted below its base asset value (e.g. you use a Swap feed or the discount for maturity causes losses)

ERC7540

Assets and TVL need to account for shares in the escrow (requested redemptions) And shares processed (maxWithdraw)

Additionally you may have gains or losses when processing a withdrawal, both of these need to be accounted

ERC4626

I listed many possible issues in `Nuanced Risk that comes from Pricing via the Pricing Oracle - With emphasis on how to Safely Price ERC4626`

You likely will need to perform DD on each integration to understand how it's work. Here's a few gotchas:

- `previewX` not accounting for losses
- `previewX` not accounting for gains (redeeming too many assets while the math seems fine, possibly giving away yield to withdrawers)
- `redeem/withdraw` locking in losses to you

Notes on fixing the issues

Some of these issues may never happen as long as you perform DD on the YieldSources

Mitigating some of these issues will likely require a custom oracle per YieldSource

And likely will require a combination of calling `previewX` `convertToX` and `X` directly (reverting after to determine the value redeemable).

A-03 Suggested Next Steps

Executive Summary

There seems to be a clear distinction between what was already considered and hardened vs what wasn't considered.

More specifically some components of the system that have key functionality within the code are not fully developed (e.g. PPS oracle).

Similarly a significant set of issues found was related to underdeveloped parts of the code.

Due to the reliance on these components the next set of reviews should have everything in scope.

High Level Architecture

Fundamentally the SuperVault has:

- A Core ERC7540 Vault with Synch Deposits
- A set of arbitrary commands (with some checks and balances) for day to day operations (using hooks)
- A external oracle that provides the PPS

The number of issues tied to the Vault was small and limited mostly to gotchas with integrations The number of issues tied to the hooks was also small and tied to the oracles The oracle itself is not in scope, nor is exactly clear what it's implementation is, however many issues are related to it and the sources available in core / other parts of periphery (out of scope for the review but superficially checked), seem to imply that the code for the oracles is not fully finished.

During this review I've enumerated a set of things that can go wrong when dealing with ERC4626 and ERC7540, we've also flagged issues related to the PPS, whether due to execution losses not being properly imputed or due to value leaks.

I have a strong conviction that the process in which the PPS is generated, as well as the timings at which that's done will lead to further issues for the SuperVault, as well as a demand for continuous monitoring.

Due to the following my recommendation is that when you do proceed with a public contest, you make sure to have the entirety of the system components in scope.

It's intellectually pretty easy to assume the oracle will behave correctly, however, given the complexity of the various integrations this should not be taken for granted.

Given these considerations my recommendation for next steps is the following:

Suggested next steps

Fix all issues identified in this review Maintain the Invariant Tests and update them based on the fixes and changes

Write a separate invariant test suite for the oracles (simplified tests that only target yield source and oracle implementation, to demonstrate that the oracle is well behaved)

If you will end up having many changes to the oracles consider doing a separate review for it along with the invariant testing.

If you won't have many other changes then you should be fine to prepare for a public contest.

The specific success criteria for the oracle integration should be that if the rest of the system is safe, but the oracle has bugs, you should not proceed with yield sources for which the oracle has bugs, until all of those are resolved and reaudited.

Overall I expect that as the system reaches it's final stage of maturity there will be integration risks and possible issues, many of which will require robust documentation, a set of steps to evaluate if the integration is well behaved (see my analysis on ERC4626 integrations) and continous monitoring to ensure that the admin usage of the system is consistent with how it was invariant tested.

A-04 Nuanced Risk that comes from Pricing via the Pricing Oracle – With emphasis on how to Safely Price ERC4626

Executive Summary

Pricing a withdrawal of X assets with `ERC4626.previewWithdraw` can superficially seem correct.

However this will likely include the losses (expressed in shares) that are necessary in order for the withdrawal to happen.

This will lock a loss to the vault.

This loss should be imputed to the user.

In order to do so you have to price the ERC4626 while ignoring the losses that should be considered slippage.

Pricing ERC4626 Safely even when the Vault is not compliant

- Convert
- Preview
- Execution

All 3 aspects have to be accounted for.

Real examples

Morpho – Standard

`convertToShares` uses the up to date assets: <https://github.com/morpho-org/metamorpho/blob/380bf7bfbcb158bfb87150056d28c7a8688b642ef/src/MetaMorpho.sol#L631-L637>

Therefore it will help establish the shares necessary to perform the withdrawal. (Rounded down)

From there you'd use that amount, since it's rounded down the controller will take a small loss. The Vault may have a small gain or no gain.

Yearn – Not Standard wrt Losses

<https://github.com/yearn/yearn-vaults-v3/blob/104a2b233bc6d43ba40720d68355b04d2dc31795/contracts/VaultV3.vy#L2084-L2102>

`previewWithdraw` will convert the assets to shares rounding up. You likely would rather use `convertToShares` and round down. As to not take the small rounding loss to the vault.

Actually withdrawing and redeeming can have sizeable losses. You'd want to compute the assets that you will receive by simulating the operation offchain.

Only those assets (received by the amount of shares computed via `convertToShares`), should be given to the user. The vault should lock no loss at that point (at least at face value)

Euler Earn – Not Standard wrt Gains

Euler earn has the gotcha that it will accrue assets on withdrawal, this can actually lead to receiving more assets per the shares. Those excess assets should be kept in the SV instead of being attributed to the user.

As such you'd apply the same logic as with Morpho. Then check the delta balance and return the min between the requested amount and the amount received.

ERC7540 - Not Standard wrt both Losses and Gains

For ERC7540 you'd compute the shares necessary (convertToShares) at face value, rounding down (as to impute the small loss to the controller).

When claiming via withdraw (withdraw(maxWithdraw)), you'd then only grant the controller at most the assets they should have received at the older exchange rate (keep the yield in the SV), if any loss happened then the controller would receive even less.

A-05 Superficial discussion on Risks and Complexities of PPS being determine by a group of validators

Executive Summary

PPS update is subject to the following risks:

- N submitters pushing an invalid price
- Submitters needing to coordinate to reach a consensus on a price

Determining if a price is valid

This in it of itself is not a solved issue as ultimately determining the right price (information) is a function of identifying the source more so than performing the computation necessary to achieve it.

Due to this, most often you are left with 2 options

Self guided discovery

Where price submitters identify sources autonomously, with the advantage that they can quickly adapt to new price sources and the downside being that it makes it more complex to validate whether the source is valid or acceptable

Hardcoded sources

Have the upside that verification can be mostly automated, the sources likely would be vetted and therefore could be more trusted. The clear downside is the continuous need for updating the sources as well as needing to revalidate their reliability.

Hardcoded onchain sources

Provide an additional level of guarantees as we can use the value onchain thanks to storage proofs: <https://github.com/curvefi/storage-proofs> (Not sure if audited)

The process of using these proofs has been discussed for years and works reliably, IIRC the only downside is needing to store each block blockhash to ensure proofs can then be referenced against that.

This would allow you to effectively prove all prices, and have a dispute process that is not based in game theory but rather in actual code.

Following the same reason, you can likely achieve gas savings (with the downside being computation complexity and risk of bugs) by using zkProofs to prove the storage values in aggregate.

Resolving Price Discrepancies

In lack of a fully provable process you likely will have to rely on game theory to provide some level of security.

Here's a few ideas:

- Identify aberrant prices through weighted mean -> Compute the mean for all prices (weight it based on trust / historical accuracy if you deem that useful), slash submitters that propose aberrant prices (requires the prices to be onchain)
- Select a proposer on each round -> Give more power to one or more proposers chosen at random (similar to how Ethereum works), they will have the ability to slash submitters
- Grant a centralized entity the ability to slash -> Puts all the power (and risk) onto one entity

A-06 Collection of Admin Related Risks

Manager can bypass the emergency withdraw when at least one hook allows for swapping

Anytime a hook allows for swapping of assets The manager can perform the following to funnel all funds out of the Vault:

- Setup a swap with 99.99% slippage
- Imbalance the pool to cause said loss (via flashloans)
- Perform the swap and have the check pass (since it's supplied by the manager)
- 99.99% of funds have been stolen from the vault

This can happen for 2 reasons:

- Malicious Manager
- PK Leak of the Manager

Manager can mistakenly move funds that are reserved for redemptions back into a hook

Unlocking these funds should be possible thanks to `executeHooks`

Primary admin can bypass all slippage checks for redemptions

The admin can call: <https://github.com/superform-xyz/v2-periphery/blob/3e58774e1acbc970b318b10c9aca529fad325421/src/SuperVault/SuperVaultStrategy.sol#L405-L411>

```
function updateMaxPPSSlippage(uint256 maxSlippageBps) external {
    _isPrimaryManager(msg.sender);
    if (maxSlippageBps > BPS_PRECISION) revert INVALID_MAX_SLIPPAGE_BPS();
    _maxPPSSlippage = maxSlippageBps;
    emit MaxPPSSlippageUpdated(maxSlippageBps);
}
```

And set a slippage of 99.99% making the PPS check ineffective

A-07 Collection of Operative Risks

Executive Summary

This is a collection of risks that Vault Managers should be aware and plan around.

Execute Hooks can cause losses (or gains) to the vault but won't automatically update the PPS

This means that there can be a delay between the totalAssets changing and the PPS reflecting that.

This will allow MEV actors an opportunity to arbitrage the discrepancy

Sufficiently high fees as well as batching rebalancing with price updates will prevent this.

The Vault may be using a stale price

Due to blocks being full the oracle may not be updated in time resulting in leaking value

Price updates may be frontrun to unfairly steal yield gains

Asynchronous vaults that allow synchronous operations carry the risk that actors can perform operations with new information while the oracle has yet to update.

A classic example is trying to frontrun a loss (mitigated by using the asynchronous flow for redemptions)

However for gains, the vault is gameable, all gains that are not vested in some way can likely be arbitrated by frontrunner

The Execution of a withdrawal may be sandwiched to cause higher losses than intended

Some vaults may be rebasable, the admin should ensure that each fulfilled redemption receives some level of assets that is consistent with their offchain simulations

Slippage tolerance should be paired with a record of losses to the VaultStrategy

Since the strategy can lose funds in performing a withdrawal, these losses should also be monitored by the manager. Perhaps by adding a b4 / after check in their Multisig Scripts

Order of Operation can create MEV opportunities for the Manager

Operating on the Vault consists of:

- Processing deposits (with a PPS and a Minting fee)
- Processing redemptions (with a zero or non zero slippage, depressed or over estimated pps)
- Investing / Divesting and Rebalancing Idle funds

Deposits are permissionless but the update of the PPS is not.

Rebalancing may happen infrequently, but could also be done often, rebalancing itself could cause losses or small gains (although unlikely), and based on the pricing methodology (e.g. mark to market vs preview swap), could result in changing the PPS in both directions.

For similar reasons claiming rewards should lead to an increase in the PPS, but for technical reasons (slippage, poor execution, 2 quick PPS updates), this can also lead to PPS decreasing after auto-compounding rewards.

Redemptions carry many MEV opportunities, since redemptions could be fulfilled:

- With price that ignores rewards (increases PPS to the Vault)
- At PPS (causes only rounding error level changes to the PPS)
- With a gain to the redeemer (due to technical mistakes, causing a loss to the vault)

The combination of these opportunities for the manager gives them the ability to generally grant or withhold an update to the PPS worth of yield to actors interacting with the vault.

In extreme scenarios:

- Loss to the Vault
- Massive gain to the Vault

These can be significant.

Because of the predictability yet nuance of this MEV opportunities, my recommendation is that each manager explicitly state their "algorithm", they should provide a clear explanation as to how they would handle these (and emergency scenarios), as to provide clarity to what would happen.

It's worth noting that these MEV privileges are shares with ETF issuers, it may be worth reviewing how CeFi counterparties handle these scenarios, hopefully in a transparent way.

A-08 Some gotcha tied to PPS being aberrant and vault paused - Likely totalAssets would then also be a dangerous value to read for integrators

null

A-09 Incomplete List of Gotchas related to the Integration Targets of the SuperVault

Executive Summary

Below are listed some notes tied to integration risks, for the integration targets provided.

Because the scope of this review didn't include all Oracles and the algorithms used to compute the PPS, we are unable to enumerate many possible attacks.

However, that doesn't prevent us from discussing them at a high level.

[p0] ERC4626 Vaults

Aave -> Bad debt may not be accrued. ERC4626 Compliant. Fluid -> May rebalance out of asset, pricing of LP token can be inaccurate, may incur additional swap fee to rebalance back to asset. Rebalancing should in general cause losses (I recall seeing some discussions on twitter), the Fluid token subsidizes these losses. Morpho -> Bad debt may not be accrued. Morpho markets with low liquidity can be grieved (doesn't apply to high liquidity markets). ERC4626 Compliant. Yearn -> Can cause DOS due to requiring locking in a loss. Technically ERC4626 Compliant, but has additional functions that are not compliant.

[p0] ERC5115 Vaults - Pendle Ethena

The Pendle vault allows to use the asset and the yieldToken for deposits and withdrawals.

Some tokens are not redeemable without cooldown (e.g. sUSDE), this means you will only be able to withdraw to sUSDE.

This limitation means the SuperVault will have to be denominated in sUSDE as well.

[p0] ERC7540 Vaults

Centrifuge -> Can take time before funds are unlocked. Can have delay in updates or the updates can be incorrect | Some small rounding errors | May not update losses in real time | Can take on losses during withdrawals and those losses will be imputed to the caller.

1Inch & Odos

Router risk, mitigated by having no allowance except when swapping

Admin may be able to bypass validation if they are able to use P2P swaps

Merkle Incentives

Anyone can claim on behalf of the recipient, monitoring scripts will have to check both the balance in merkle and the idle balance in the Strategy

Pendle & Spectra

Risk of Slippage. Possible risk of losing tracking of the asset by splitting. Possibly receiving less yield when splitting and selling PY, however I don't think you can lose asset in this way. Likely to require expiration by expiration, update of hooks to ensure it works properly.

Loans

Morpho -> For markets in which the price is hardcoded (e.g. Ethena hardcoded to 1 USD), the risk is actually shifted to the lenders. This is good for you (borrowers) as they carry the risk of the underlying asset losing value relative to what you borrow. It's worth noting that since you'd likely:

- Deposit PT sUSDE
- Borrow USDC
- Sell USDC for USDE and stake

You'd also be impacted in relative terms, but since you'd denominate the vault and strategy in USDE the vault itself would not lock in a absolute loss. (although USDE could lose value vs USD)

Staking

Gearbox ->

Source Checked: <https://etherscan.io/address/0x9ef444a6d7F4A5adcd68FD5329aA5240C90E14d2#readContract>

Cannot deposit nor act on someone elses behalf. Safe afaict.

Fluid -> TODO Check | Can have a DOS of up to 4 days if the Fluid DEX hits an edge case:
<https://docs.fluid.instadapp.io/cantina-audit-dex.pdf>

Source checked: <https://etherscan.io/address/0x2fA6c95B69c10f9F52b8990b6C03171F13C46225#code>

Cannot deposit nor act on someone elses behalf. Safe afaict.

Yearn -> No particular concern -> Smart Contract Risk / Underlying strategy risk.

Source checked: <https://github.com/yearn/tokenized-strategy-periphery/blob/0f0c380a32342772fb17709dea9b7cc3a8a0162f/src/Bases/Staker/TokenizedStaker.sol#L390>

Cannot deposit nor act on someone elses behalf unless approved. Likely won't be approved. Safe afaict.

E-01 Deposits and Withdraw will benefit by having a Bid and Ask Pricing

Impact

Deposits will have their shares computed as follows: <https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L159-L162>

```
uint256 pps = getStoredPPS(); /// @audit Stored PPS = Gain in yield is likely
if (pps == 0) revert INVALID_PPS();
sharesNet = Math.mulDiv/assetsNet, PRECISION, pps, Math.Rounding.Floor);
if (sharesNet == 0) revert INVALID_AMOUNT();
```

Redemptions are currently processed using the same `getStoredPPS` .

<https://github.com/superform-xyz/v2-periphery/blob/8089ede43dc3eff0b53bfc3ee8e03a549def3199/src/SuperVault/SuperVaultStrategy.sol#L305-L306>

```
_processRedeemFulfillments(args.controllers, controllersLength, processedShares, currentPPS);
```

This creates more risks than necessary as it opens up to MEV actors exploiting inaccuracies in pricing directions whenever present.

POC

- A redemption is processed
- The price is purposefully kept low to underpay shares (as to not create any economic leak)
- The same price can now synchronously be used for deposits
- Deposits are giving away some free yield

And the opposite case

- A deposit is processed
- The price is slightly higher than intended to combat MEV at the deposit level
- The manager ignores this and fullfills redemptions
- The redemptions have leaked value from the SuperVault to the controllers

Mitigation

Having a separate bid and ask, deposit and redeem prices, where deposits are overprice and redemptions are underpriced will mitigate this issue.

It will also avoid the need to have higher fees as to combat arbitrage.

Additional Services by Recon

Recon offers:

- Audits powered by Invariant Testing - We'll write your invariant tests then perform an audit on your code.
- Cloud Fuzzing as a Service - The easiest way to run invariant tests in the cloud - Ask about Recon Pro.
- Invariant Tests writing - An engineer will write Chimera based Invariant Tests on your codebase.